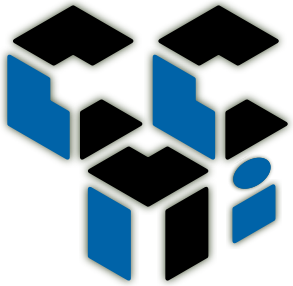
OntoUML: Základy, rigidní a anti-rigidní sortály





FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

BI-KOM: Konceptuální modelování #2

doc. Ing. Robert Pergl, Ph.D. (robert.pergl@fit.cvut.cz)

Pozn.: Pro přípravu přednášky byly mj. využity dostupné materiály prof. Giancarla Guizzardiho.

Obsah přednášky

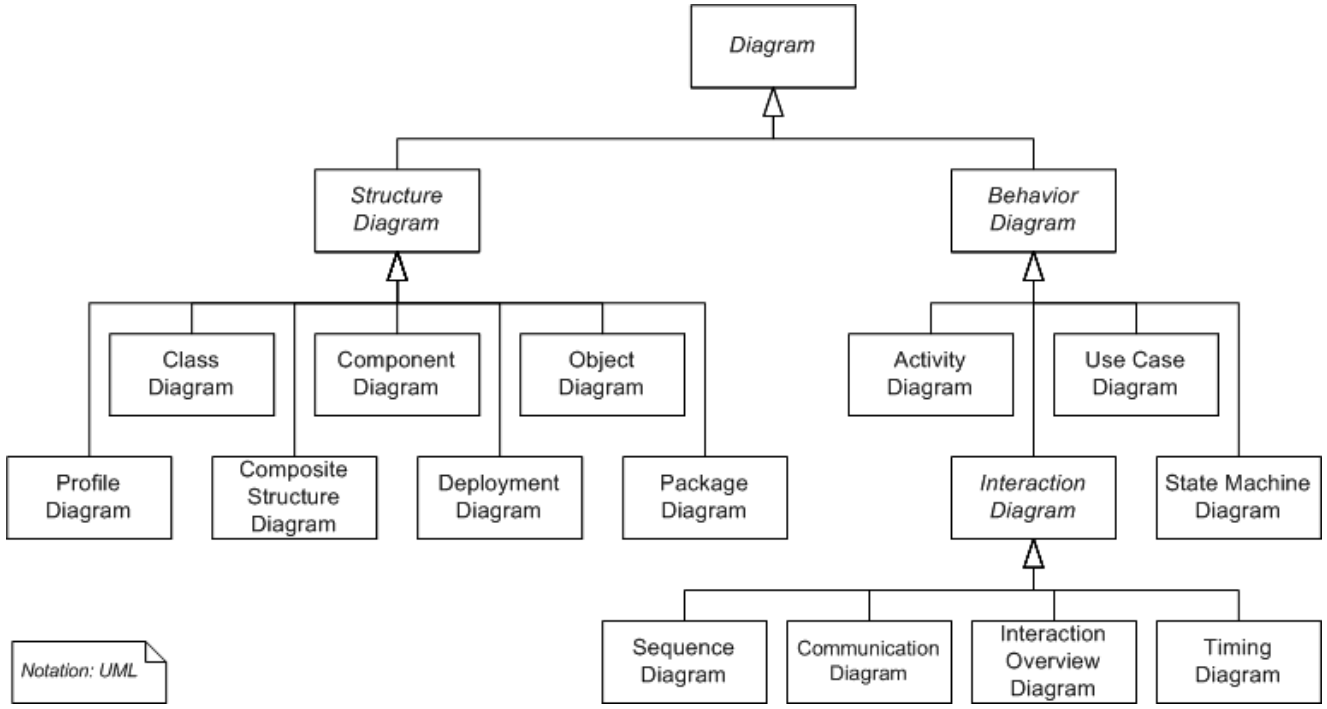
1. UML a OntoUML
2. Postup modelování
3. Základy (Onto)UML
4. Stereotypy: «Kind», «SubKind», «Phase», «Role»

UML – Unified Modeling Language

- Jazyk pro konceptuální modelování informačních systémů, elektronických systémů, výrobních systémů, business procesů, organizačních struktur, atd.
- Založen na metamodelu MOF (MetaObject Facility), definuje základní typy entit třída, instance, atribut, ...
- Umožňuje významové rozšíření pomocí tzv. *UML profilů* = definice diagramů a jemnějšího dělení typů entit
- Standard jazyka UML je spravován Object Management Group (OMG).

UML profil pro softwarové inženýrství (UML SE)

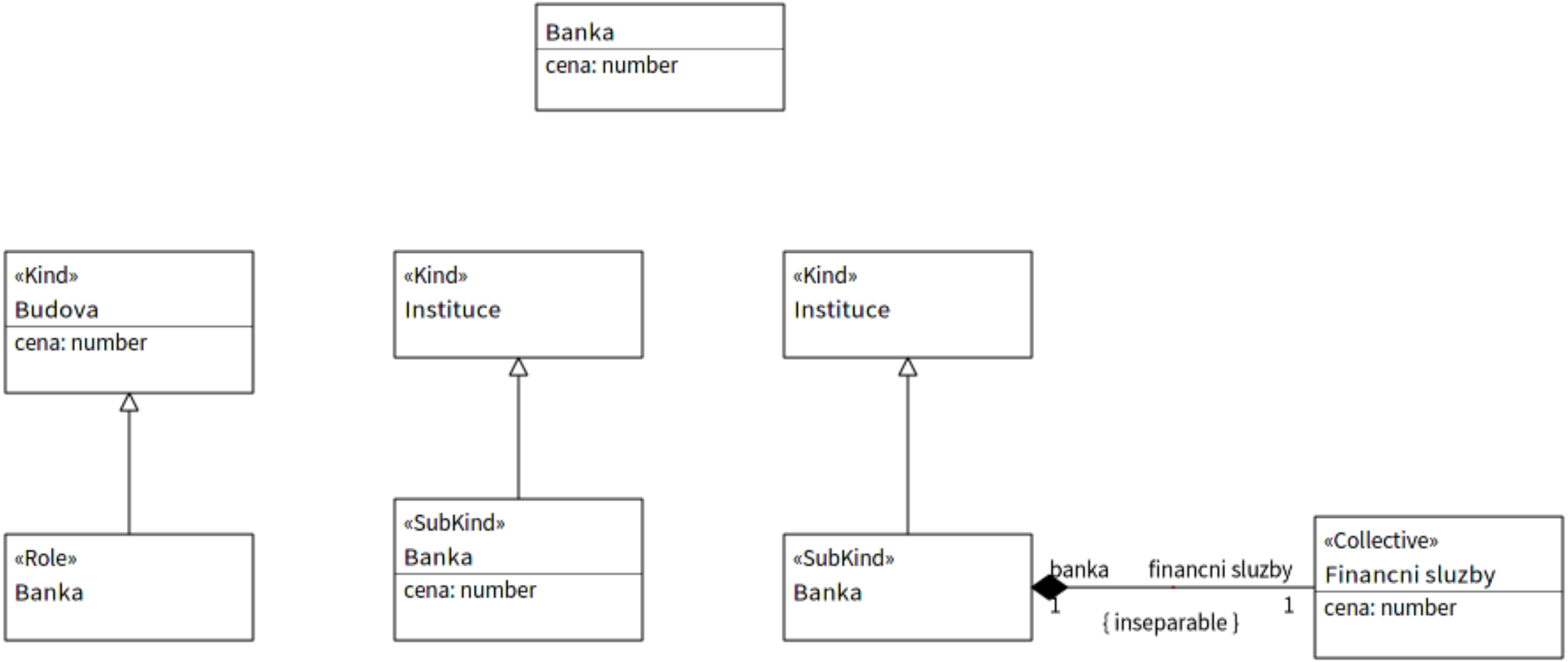
- "Grafické znázornění objektově-orientovaného systému": slouží pro objektově-orientovaný přístup k analýze a návrhu informačních systémů.
- Často bývá z historických důvodů nesprávně zaměňováno UML = UML profil pro softwarové inženýrství.



OntoUML

- Profil UML podobný diagramu tříd UML SE, ovšem s trochu jinou a přesnější sémantikou.
- Slouží pro ontologické konceptuální modelování pomocí základní ontologie UFO-A.
- Tím dosahuje vyšší úrovně výrazové přesnosti a bohatosti pro strukturální modelování než UML.

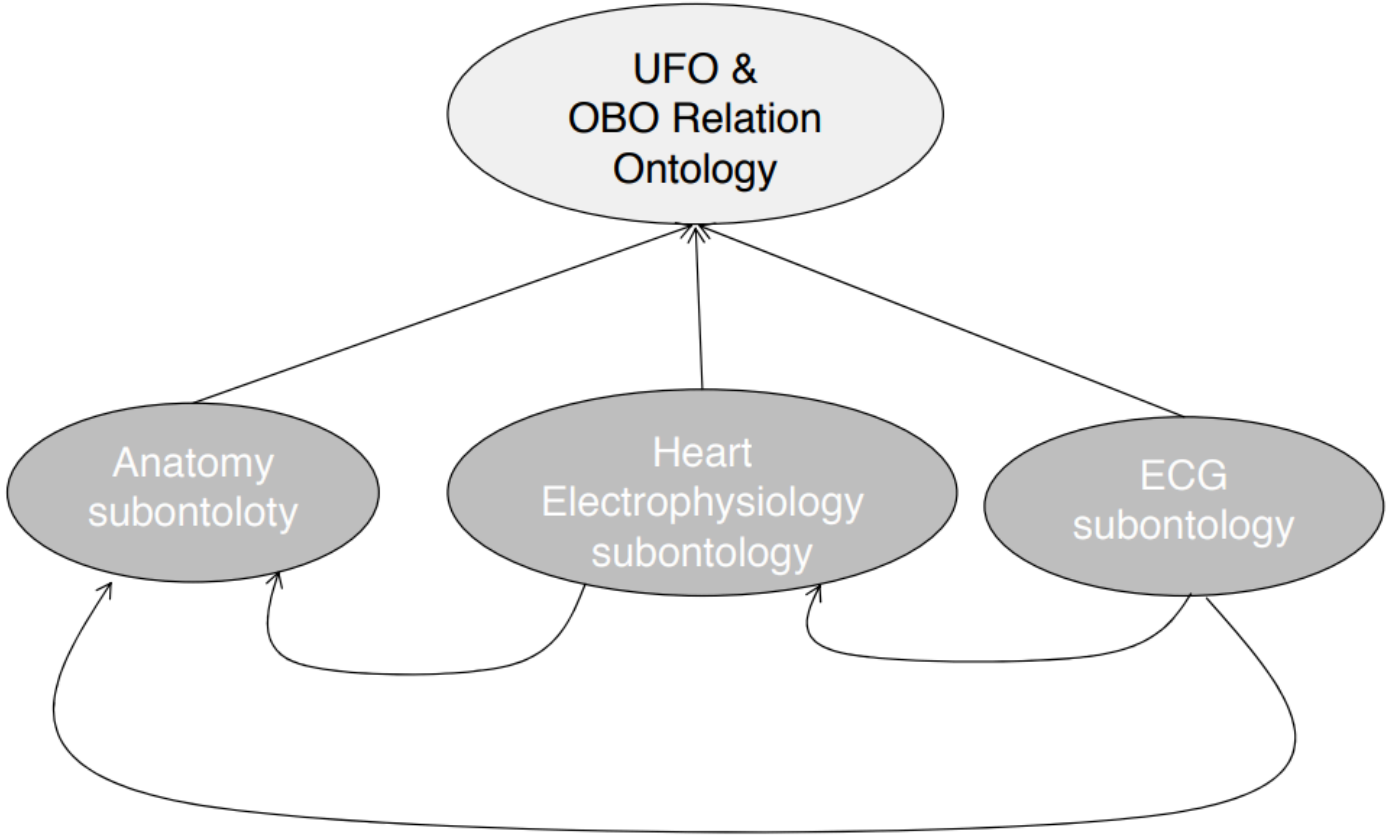
Příklad: "Cena banky" UML vs. OntoUML



Postup konceptuální analýzy: Identifikace účelu a cíle

- Označováno též jako **Requirements Engineering**.
- Vymezení rozsahu ontologie, cíle a aplikace: kompetence ontologie.
  - Dokumentace / reinženýring / návrh informačního systému / ...
  - Identifikace otázek, na které chceme znát odpovědi (využití i při validaci).
- Klíčový aspekt, jelikož vícero (**správných**!) ontologií může být vytvořeno pro jednu doménu vzhledem k různým účelům.
- Pokud je doména příliš komplexní, použijeme techniku *modularizace domény*: vytvoříme vícero subontologií zachycujících různé (ideálně ortogonální) aspekty domény:

Modularizace ontologie



(OBO = The Open Biological and Biomedical Ontologies, http://www.obofoundry.org)

Krok: Ontologické konceptuální modelování

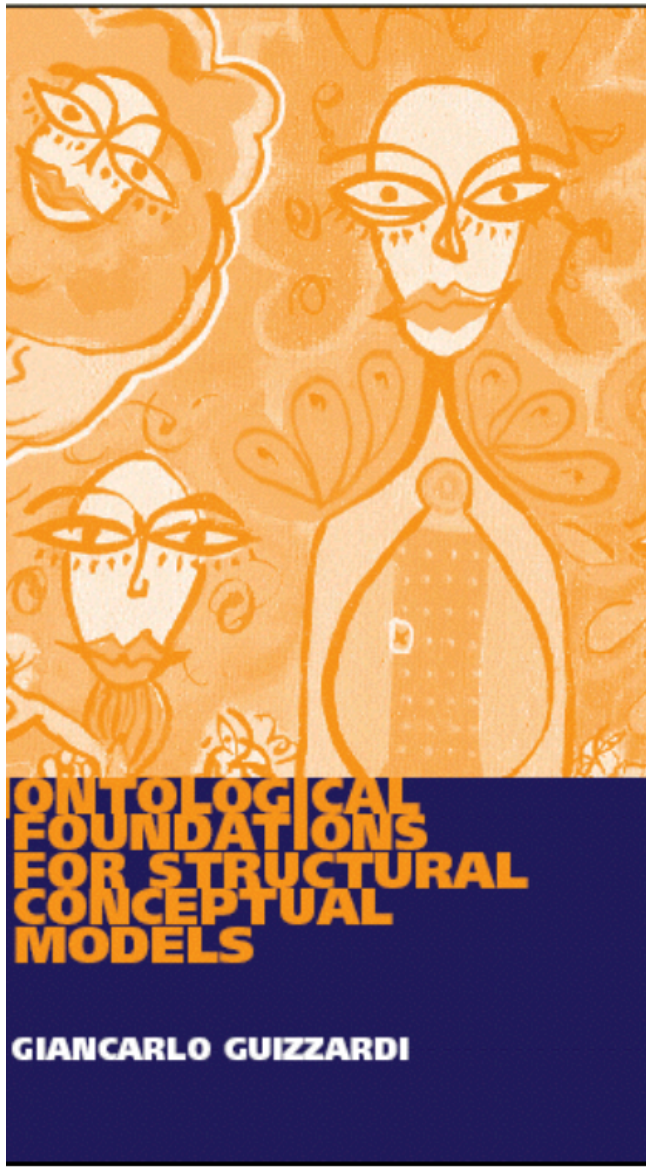
- Využití některé tzv. *základní ontologie* (**Foundational Ontology**) (1. přednáška).
- Cílem je maximalizovat výrazovost, pravdivost a konceptuální jednoznačnost.
- Postup konceptuálního modelování:
  - Vytvoření **slovníku pojmů** dle konceptů v doméně.
  - Vytváření ontologie na základě ontologických **vzorů**.
  - Kontrola modelu (6. přednáška):
    - Verifikace modelu: kontrola syntaktické správnosti = *"Model je správný"*
    - Validace modelu se zadavatelem (doménový expert) = *"V modelu jsou správné věci"*
- V tomto kurzu budeme nyní používat základní ontologii **UFO-A** (Unified Foundational Ontology pro strukturální aspekty) a notaci **OntoUML**.

Krok: Transformace na implementační model

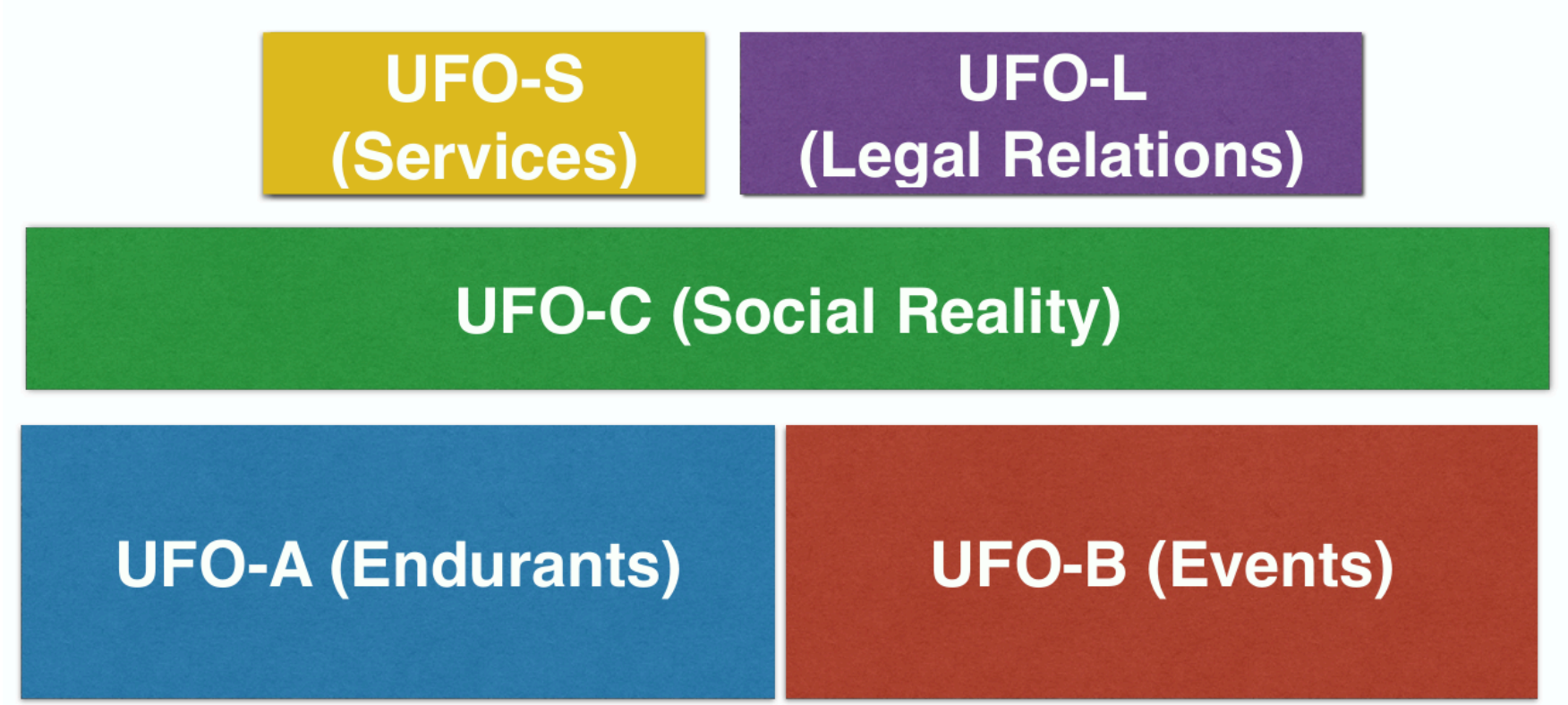
- V případě využití pro softwarové inženýrství je ontologický konceptuální model možné transformovat na různé implementační modely, typicky relační nebo objektový (7. přednáška).
- Tento přístup se obecně nazývá **Model-Driven Engineering**. Konkrétním příkladem v SI je **Model-Driven Architecture** (OMG).
- Konceptuální modely ale lze využít i pro ne-softwarové úlohy, např. ontologickou klarifikaci, komunikaci a návrh/reorganizaci podniku.

Unified Foundational Ontology (UFO, 2005)

- Vytvořená prof. Giancarlem Guizzardim a jeho týmem.
- Zapracovává myšlenky z ontologií GFO, DOLCE a Ontology of Universals (OntoClean) do jedné ucelené základní ontologie.
- Používána k analýze, re-designu a integraci ontologických konceptuálních modelů v různých komplexních doménách.



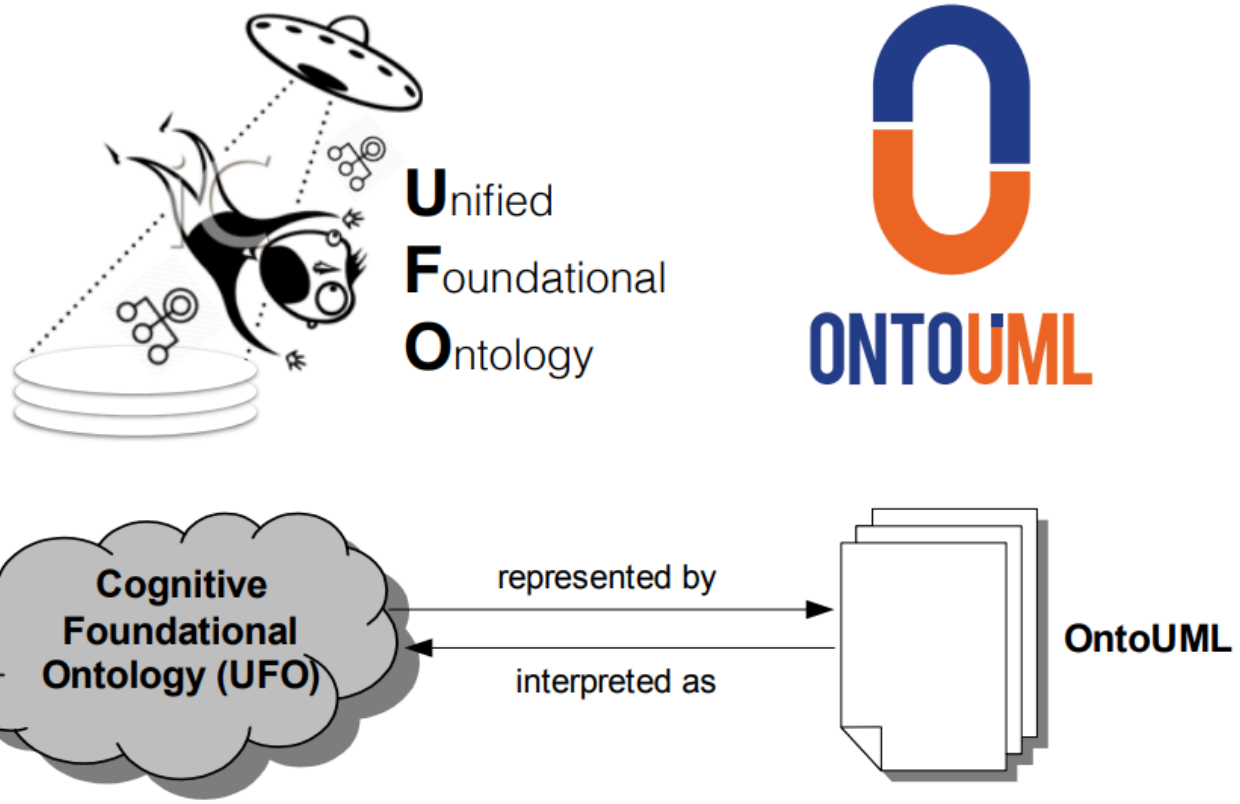
Unified Foundational Ontology



- **UFO-A**: Strukturální aspekty (entity, vlastnosti, vztahy)
- **UFO-B**: Behaviorální aspekty (chování, události, kauzalita)
- **UFO-C**: Sociální aspekty (agenti, akce, organizace)

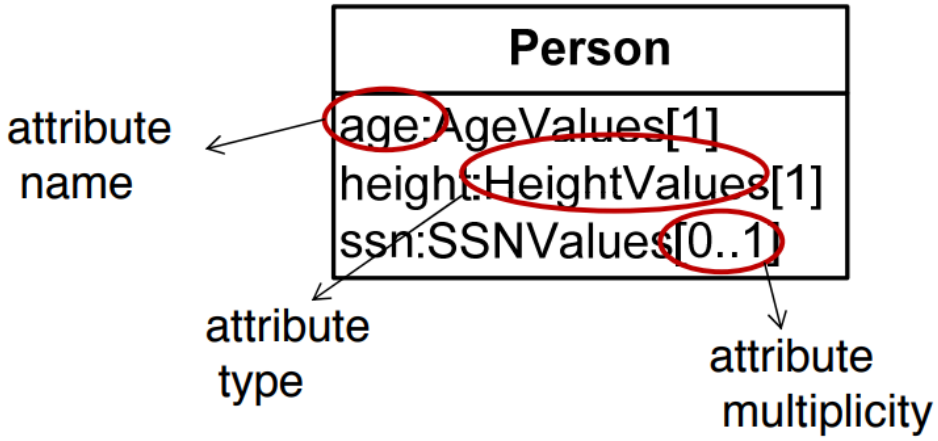
OntoUML (ontouml.org)

- UML profil založený na UFO pro *Ontology-Driven Conceptual Modelling*



Typ entity a atributy

- Typy entit v OntoUML vychází z UML koncepce tříd jakožto **popisu společných vlastností sdílených určitými entitami**.
- Entity obsahují hodnoty atributů definované svým typem v souladu s charakteristikou atributu, např. typem a multiplicitou.
- Atributy reprezentují (více či méně *intrinsické* = vlastní, vnitřní) vlastnosti sdílené entitami.



Typy atributů

String – textový řetězec: `"Petr"`

Integer – celé číslo: `10`, `-5`

Float/Real – desetinné číslo: `10.1`, `2e3`, `-1.3e-3` (případně s desetinnou čárkou)

Percent – procento: `10%` či `0.1`

Boolean – logická hodnota: `true`, `false`

Date – datum: `2.6.2012`, `6/2/2012`, `2 Jan 2012` (dle národního prostředí a preferencí)

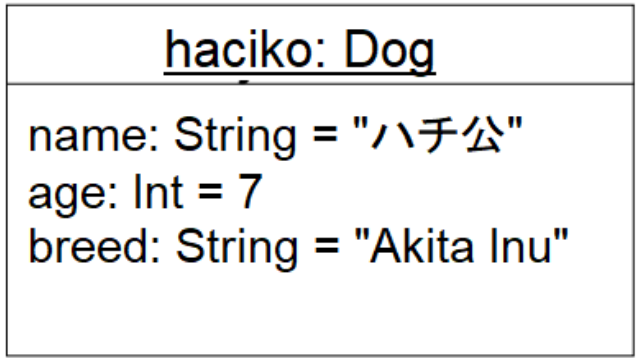
Time – čas: `21:00`, `1:30pm` (opět dle národního prostředí a preferencí)

Timestamp – časové razítko: přesná identifikace okamžiku = datum a čas s vysokou přesností (milisekundy)

Currency – peněžní částka v určité měně: `56,00- Kč`, `$102`, apod.

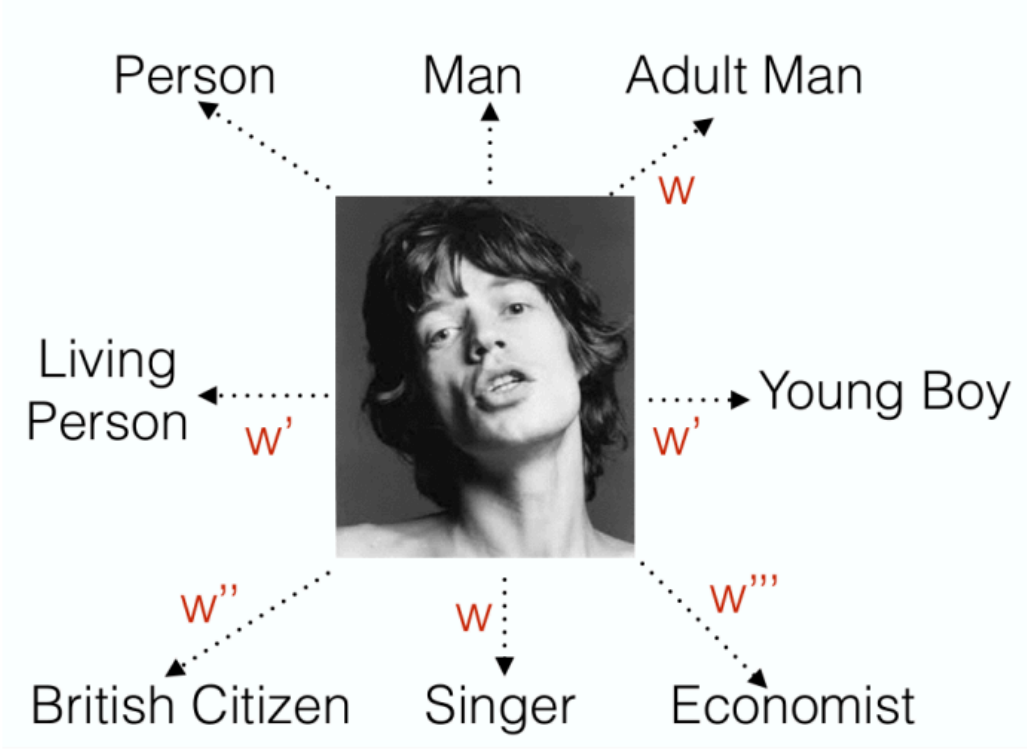
Entity

- Entity mají již konkrétní hodnoty atributů.
- Příklady jedinců pomáhají při validaci, simulaci a prototypování.
- Diagramy OntoUML specifikují **typy**; jedinec níže ilustruje jejich význam, nejde o objektový diagram OntoUML.



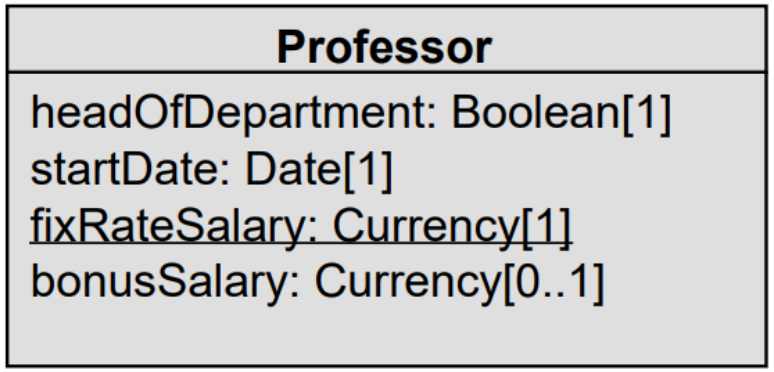
Instance v OntoUML

❗ ❗ ❗ **POZOR**: na rozdíl od UML ale není vazba třída-instance 1:1



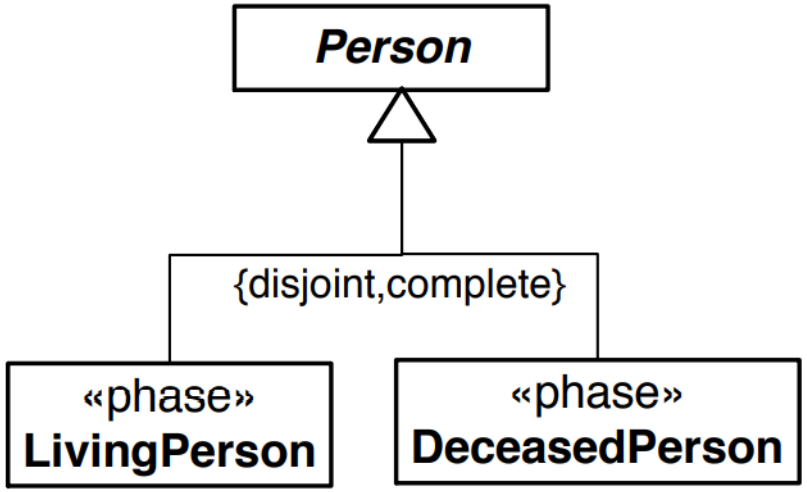
Atributy typů entit

- Atribut typu entity vyjadřuje vlastnosti, které jsou společné všem entitám, jsou tedy vlastností typu entity a na úrovni jednotlivých entit jsou konstantou.
- Syntakticky odpovídají *třídním atributům* v UML - značíme podtrženě:



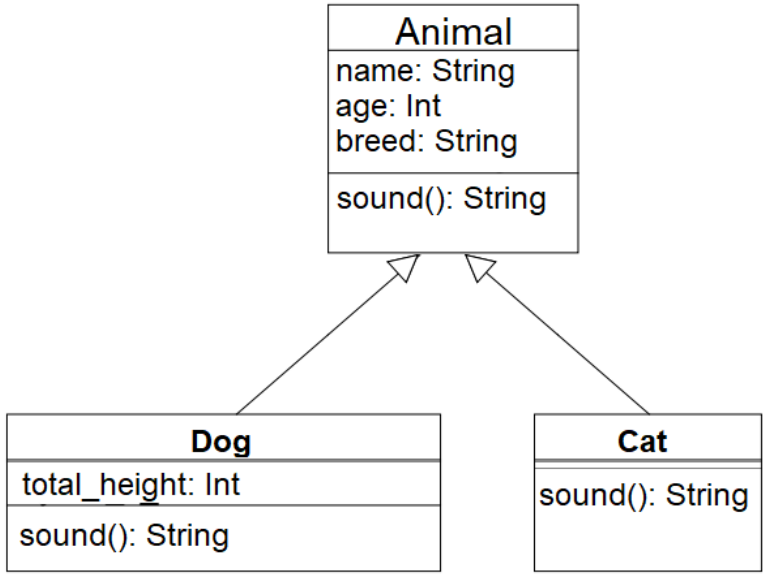
Abstraktní typy entit

- Abstraktní typ entity nemá v realitě samostatné instance. (V UML odpovídá abstraktní třídě)
- Instanci tvoří jen spolu s alespoň jedním svým podtypem.
- Abstraktní typ entity se značí názvem psaným kurzívou.



Metody

- UML a objektový model obsahují koncept metody beroucí vstupní parametry, měnící stav objektu a produkující výstupní hodnotu.
- OntoUML (UFO-A) nevyužívá, pro modelování chování slouží UFO-B.



Specializace (dědění)

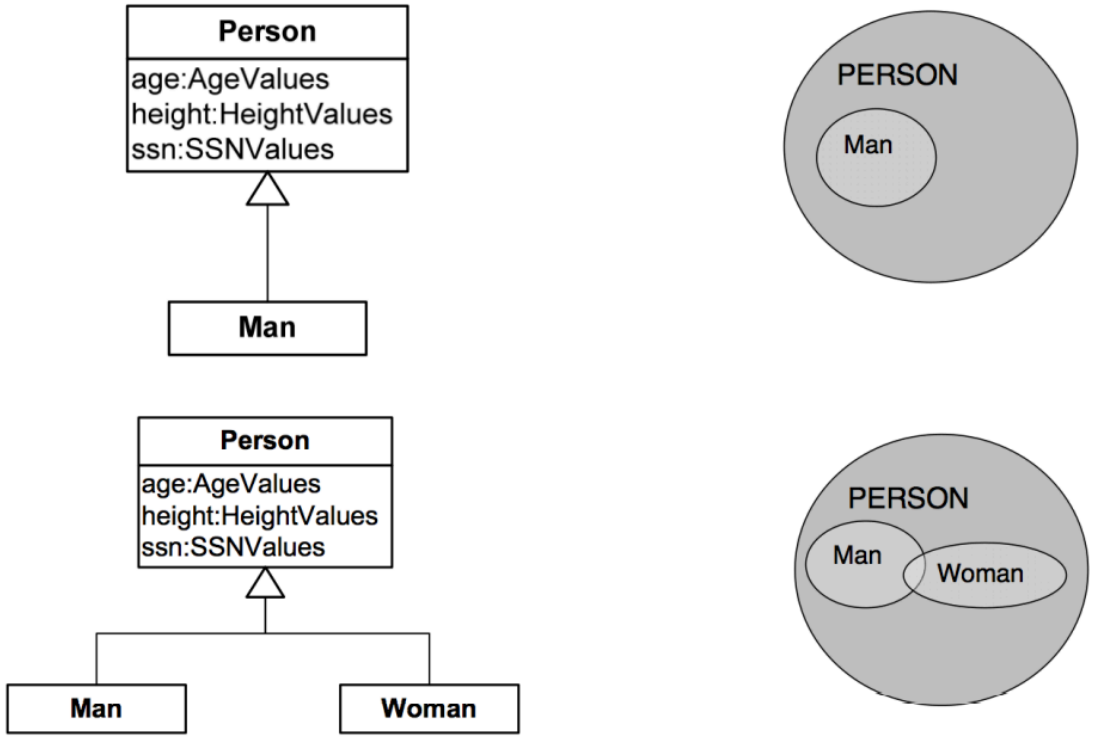
- Třídy mohou být navzájem ve vztahu specializace = taxonomická struktura.
- Všechny vlastnosti třídy (atributy a vztahy) se dědí přes řetěz specializace.
- Anti-symetrická, tranzitivní relace:
  - Pokud B je specializací A, potom A nemůže být specializací B.
  - Pokud B je specializací A a C je specializací B, potom C je specializací A.

Dědění z hlediska množinové sémantiky

- Třídy sdílející společný nadtyp mohou být sdruženy v tzv. množině nadtypu (*Generalization Set*).
- Sémantika množiny nadtypu může být upřesněna meta-atributy `complete` a `disjoint`.
- `complete` = každá instance nadtypu je také instancí alespoň jednoho z podtypů.
  - V UML terminologii odpovídá `isCovering = true`
- `disjoint` = neexistuje instance, která by byla instancí více než jednoho z podtypů.
- Pozn: Tento konstrukt obsahuje i UML, ale v OntoUML je více využíván.

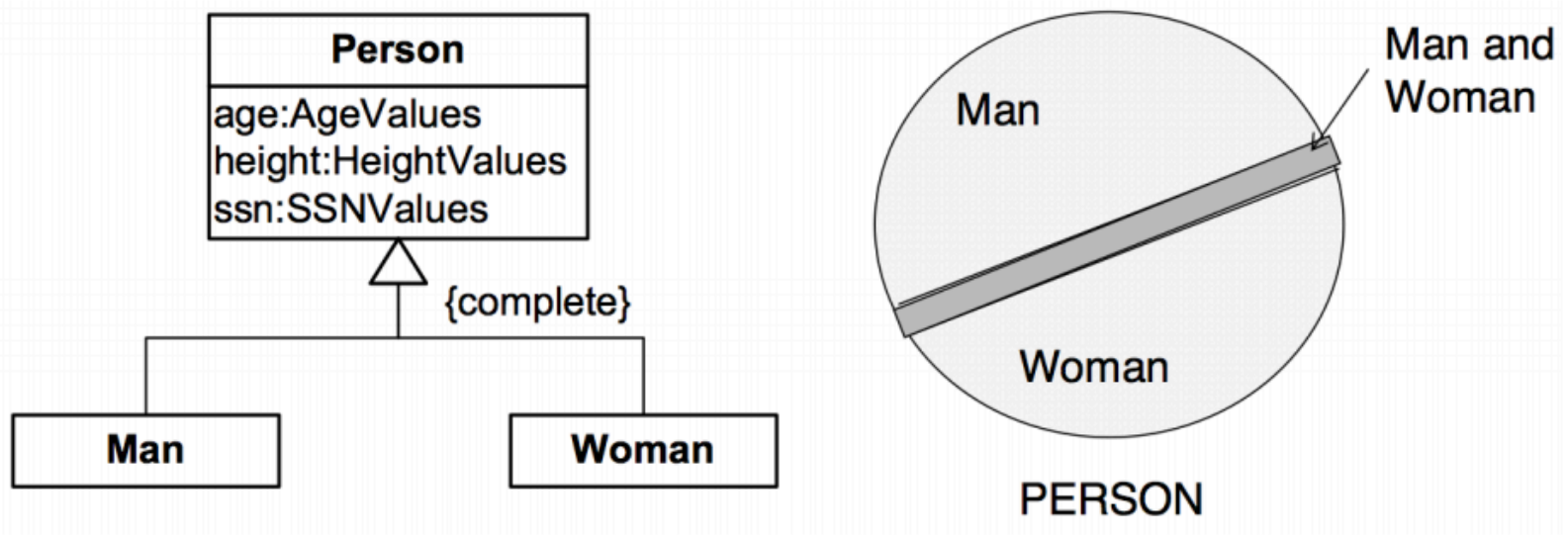
Generalization Set: bez `disjoint` a `complete`

- Mohou existovat instance pouze nadtřídy, podtříd, i jejich kombinací.
- Instance může být Žena, Muž, obojí, nebo jen Člověk.



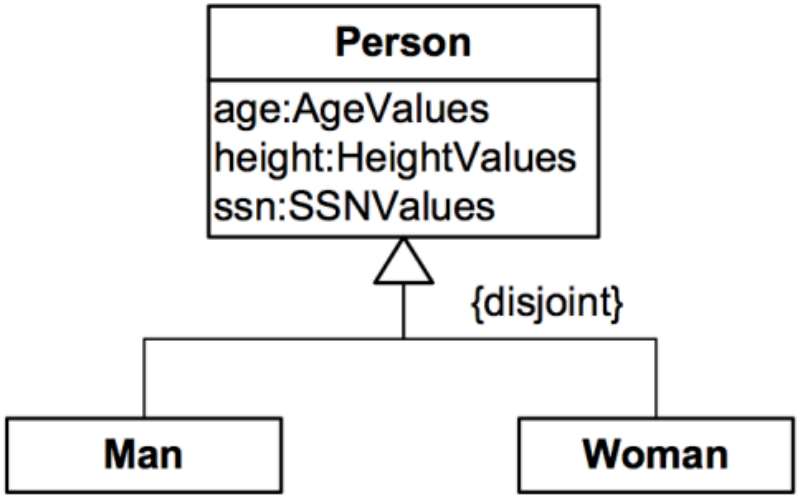
Generalization Set: jen `complete`

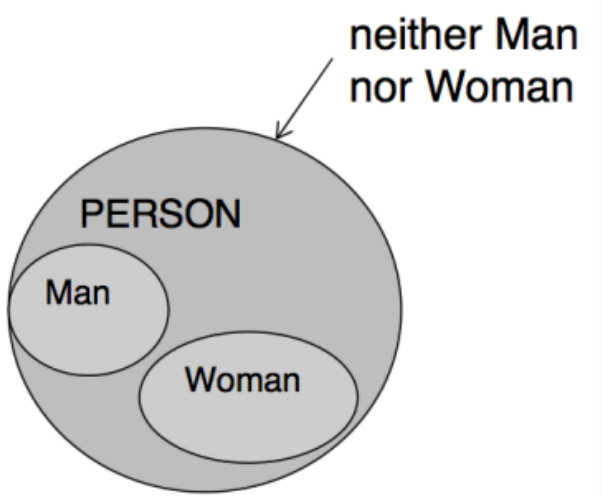
- Mohou existovat instance podtříd a jejich kombinací (nadtřída je de-facto abstraktní).
- Instance může být Žena, Muž a nebo obojí.



Generalization Set: jen `disjoint`

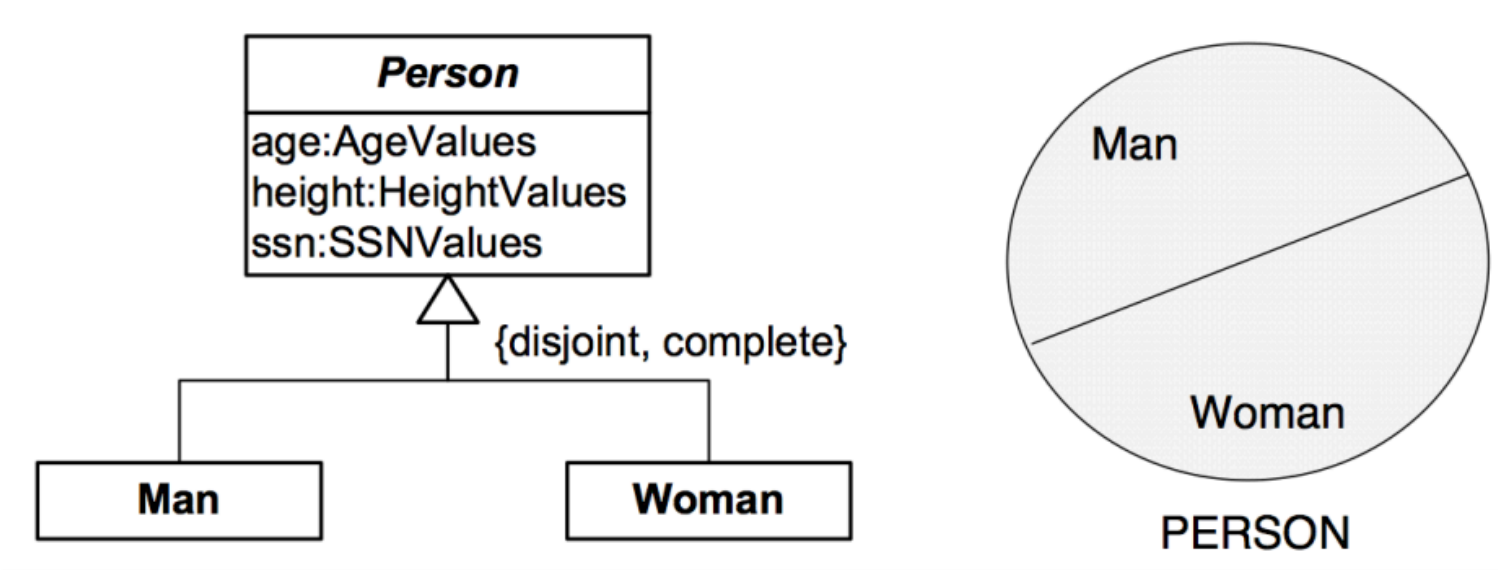
- Mohou existovat instance, které jsou instancí pouze nadtřídy nebo právě jedné podtřídy (ale ne kombinace).
- Instance může být buď Žena, Muž a nebo jen Člověk (ale ne kombinace).





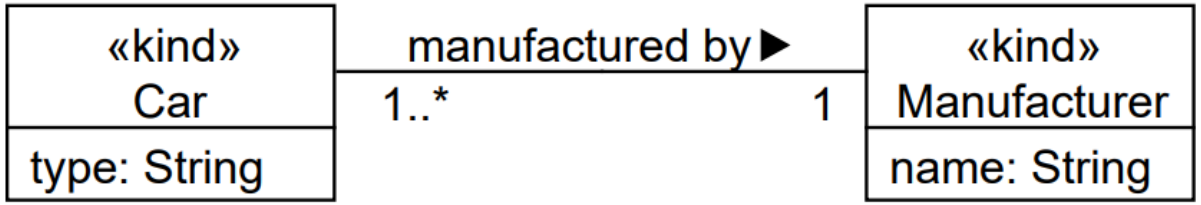
Generalization Set: `disjoint` a `complete`

- Mohou existovat instance, které jsou instancí právě jedné podtřídy.
- Instance může být buď Žena nebo Muž (ale ne obojí).



Asociace

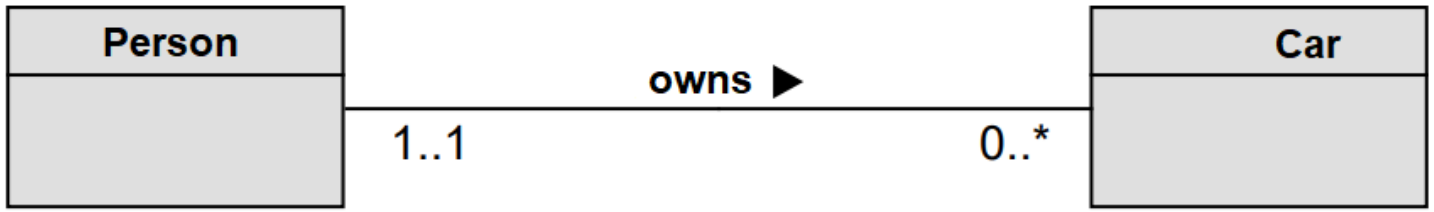
- Asociace modelují vztahy mezi entitami.
- Asociace jsou relace, které k sobě váží entity: instance relace jsou n-tice (tedy dvojice u binárních vztahů, na které se omezíme).




```
(Fabia, Skoda)
(Octavia, Skoda)
(9-5, Saab)
(900, Saab)
```

Asociace v OntoUML

- Asociace v OntoUML jsou významem i notací shodné s UML (ačkoliv samy o sobě nevyjadřují dostatečně přesně sémantiku vztahů -- stay tuned for Relators ;-).
- Asociaci specifikuje:
  - **název** = upřesňující údaj se šipkou značící směr čtení
  - **multiplicita** = povolená násobnost vztahu na obou stranách asociace
- Multiplicita se zapisuje ve směru čtení (např. *Osoba (jedna) vlastní 0..n Aut.*)!



Asociace - Multiplicita

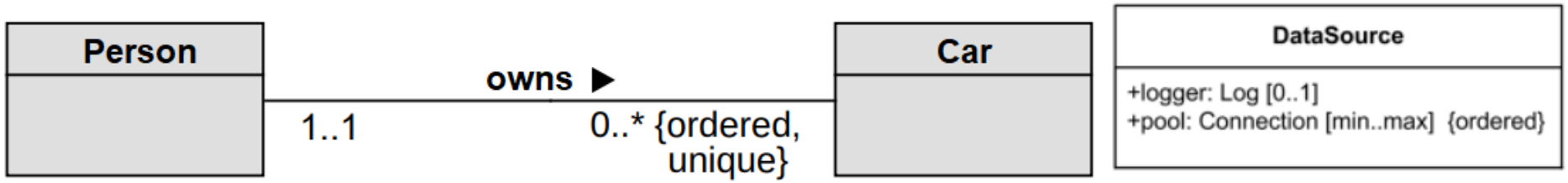
Každá multiplicita (násobnost) má:

- **spodní mez** – typicky určuje povinnost ve vztahu:
  - Osoba nemusí vlastnit žádné auto (0..).
  - Auto musí mít osobu (vlastníka) (1..).
- **horní mez** – určuje maximální počet instancí relace, kde se daná entita vyskytuje. Je buď konkretní hodnota ≥ 1, nebo ”libovolný počet“, značíme ”*“ či ”n“ (konzistentně!):
  - Osoba může mít libovolné množství aut (..*)
  - Auto může mít max. 1 osobu (vlastníka) (..1)

Pozn. ”1..1” lze psát jako ”1“ a ”0..*” jako ”*” (konzistentně!)

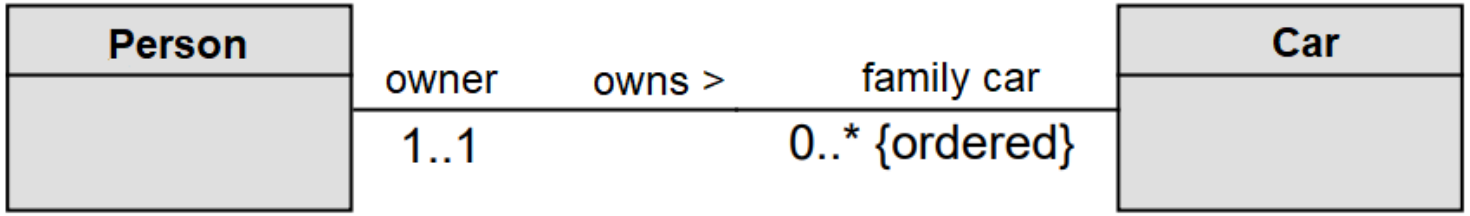
Asociace - Metaatributy

- `unordered` – neuspořádaná množina (*default*, `isOrdered = false`)
- `ordered` – uspořádané pole dle pořadí vkládání
- `unique` – prvky musí být unikátní (*default*)
- `nonunique` – prvky nemusí být unikátní, tzn. mohou se opakovat (`isUnique = false`)



Asociace - Role entit

- Mimo názvu vztahu lze navíc pojmenovat roli entity ve vztahu



Asociace - Poznámky

- Multiplicity jsou určeny problémovou doménou a jsou tedy výsledkem konceptuální analýzy. Pouze na základě pohledu na model nelze stanovit, jestli jsou multiplicity správně či chybně: k tomu potřebujeme znát určení modelovaného systému a pravidla v realitě:
  - Pokud např. bude asociace z předchozího slide modelovat databázi autoklubu, každá osoba bude vlastnit 1..* aut (podmínkou členství v autoklubu je vlastnictví auta).
  - Pokud nebereme vlastníka ”de jure“, ale jako uživatele, může jedno auto vlastnit i více vlastníků (otec, manželka, syn, ...).
  - apod.

Asociace - Typy

- Asociace je vztah oboustranný, který má název a 4 charakteristiky (spodní-horní mez, spodní-horní mez).
- Mezi entitami A a B můžeme rozlišit 3 třídy asociací:
  - **1:1** = každá instance třídy A má přiřazenu (maximálně) jednu instanci třídy B.
  - **1:N** = instance třídy A může mít přiřazeno i více instancí třídy B. Každá instance třídy B má však přiřazenu max. jednu instanci třídy A.
  - **M:N** = instance třídy A může mít přiřazeno více instancí třídy B a instance třídy B může mít přiřazeno více instancí třídy A.

Poznámky k asociacím

Pozor též na povinnost ve vztahu k časové dimenzi: povinný vztah musí být navázán **VŽDY**, tj. neexistuje žádná instance třídy A, která by nebyla v relaci s nějakou instancí třídy B:

- Na škole existuje pravidlo, že student musí studovat alespoň 1 předmět. Pokud si ale studenti zapisují předměty třeba až měsíc po zápisu, musí být multiplicita 0 (nepovinný), neboť jinak by systém vyžadoval přiřazení předmětu ke studentovi již v okamžiku vložení studenta do databáze.
- Takováto omezení nelze zachytit v ontologickém modelu a musí být vyjádřena doplňujícími pravidly (viz přednáška OCL).

Kategorie typů objektů

*„ **Those that resemble flies from a distance** is a logically possible way to group objects, but it's not how we naturally make sense of the world. No real language would have a noun for such a category... Real nouns capture something deep; they refer to **kinds** of things that are thought to share deep properties... “*

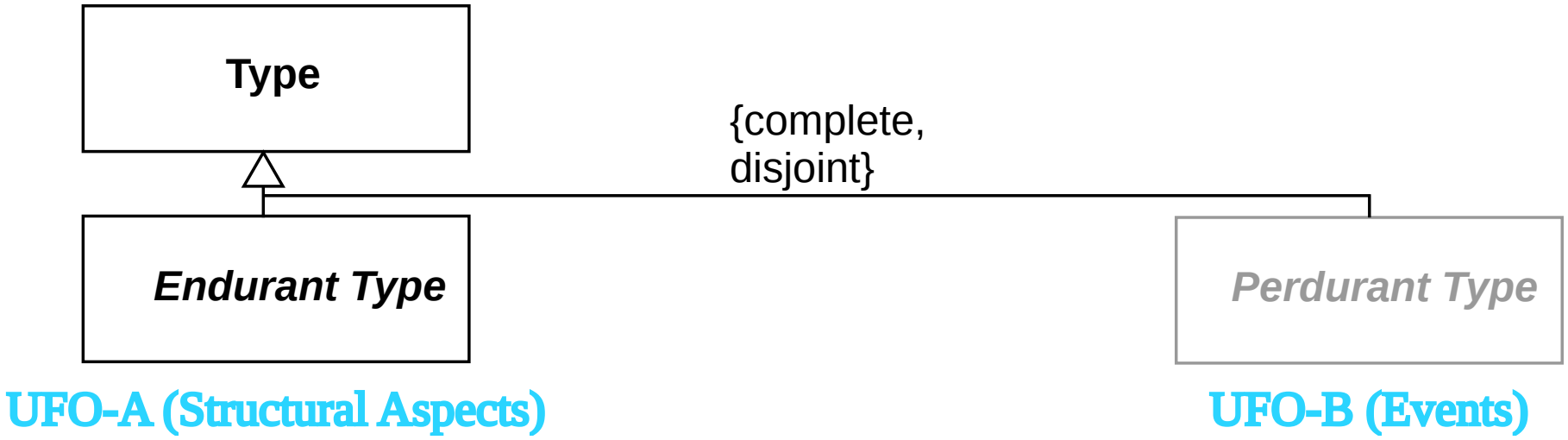
(Paul Bloom, How Pleasure Works, 2010)

Kategorie typů objektů

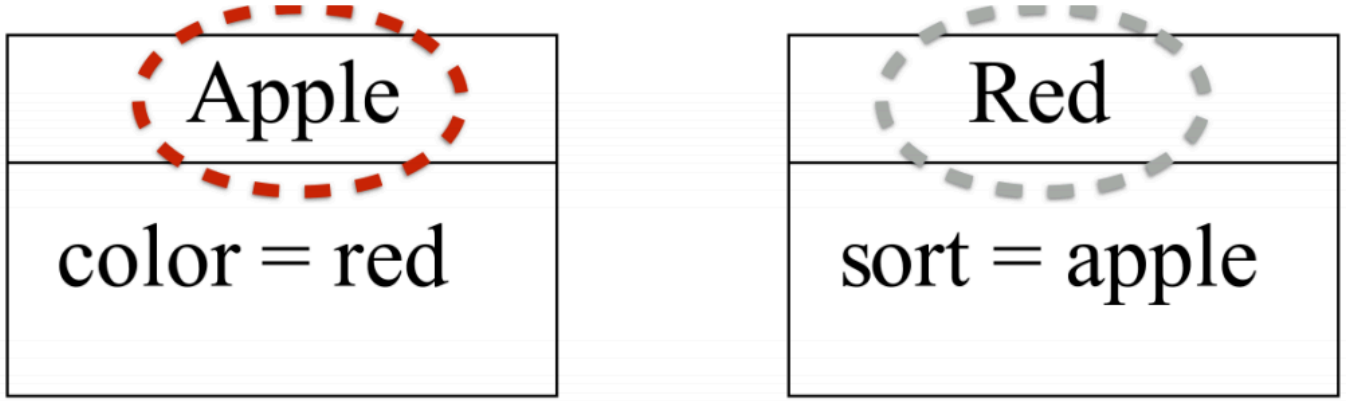
„ … As the evolutionary theorist Stephen Jay Gould put it, our classifications don’t just exist to avoid chaos, they are ***theories about the basis of natural order****. “*

(Paul Bloom, How Pleasure Works, 2010)

Kategorie typů UFO

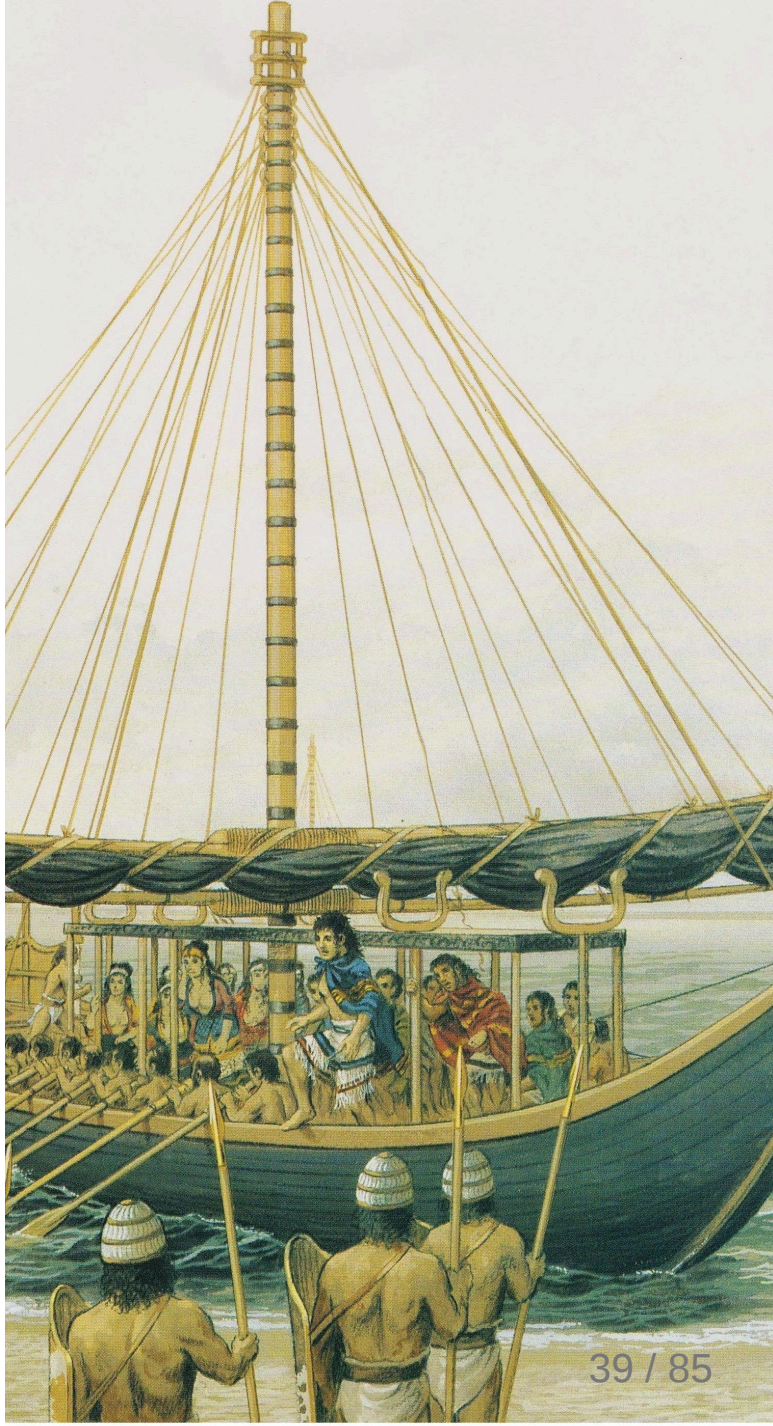


Kategorie typů objektů



1. Princip identity

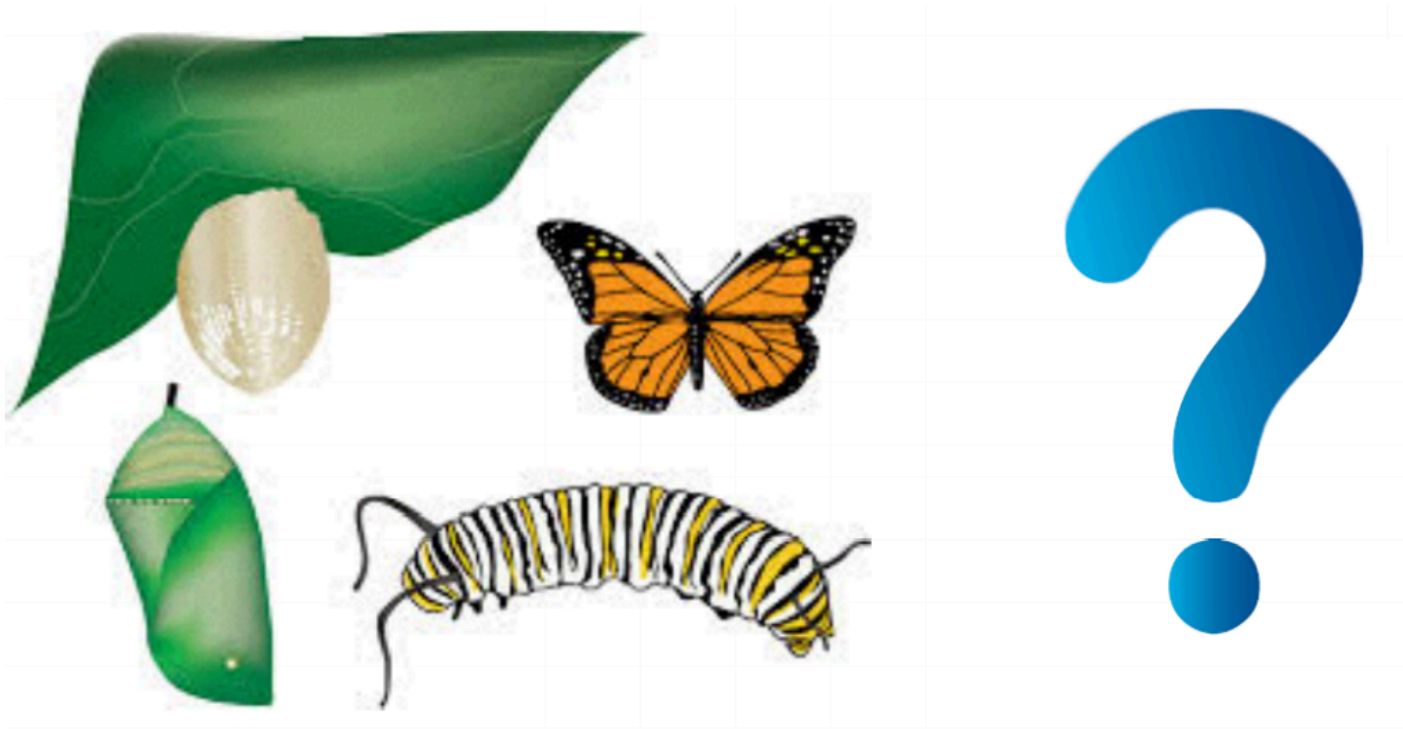
- **Princip identity** říká, že určitou věc jsme schopni identifikovat jako "onu věc" a odlišit ji od "tamté věci".
- Jelikož zrak je pro nás evolučně zásadní "rozlišovadlo", hodně souvisí s vizuální stránkou, ale někdy v průběhu života věci mohou nastat drastické vizuální/strukturální změny, které nemají vliv na identitu (a naopak).
- Jedná se opět o intersubjektivní záležitost ("dohodnutou konceptualizaci") -- viz Théseova loď.
- V programování odpovídá definování ekvivalence.
- Princip identity umožňuje *referenci* (tj. "ukázání na") a *kvantifikaci* (tj. "spočítání").



Problematika principu identity

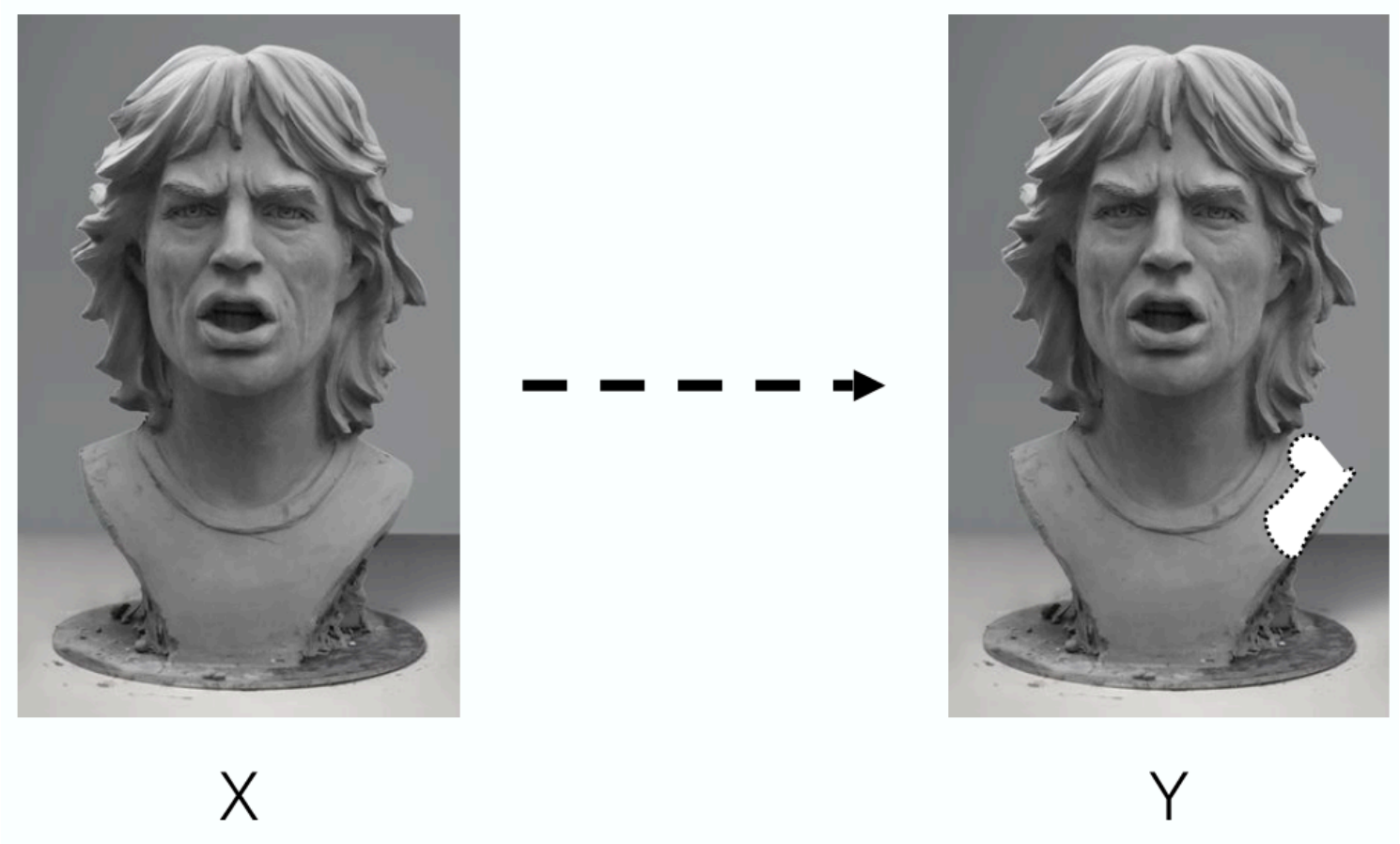


Problematika principu identity

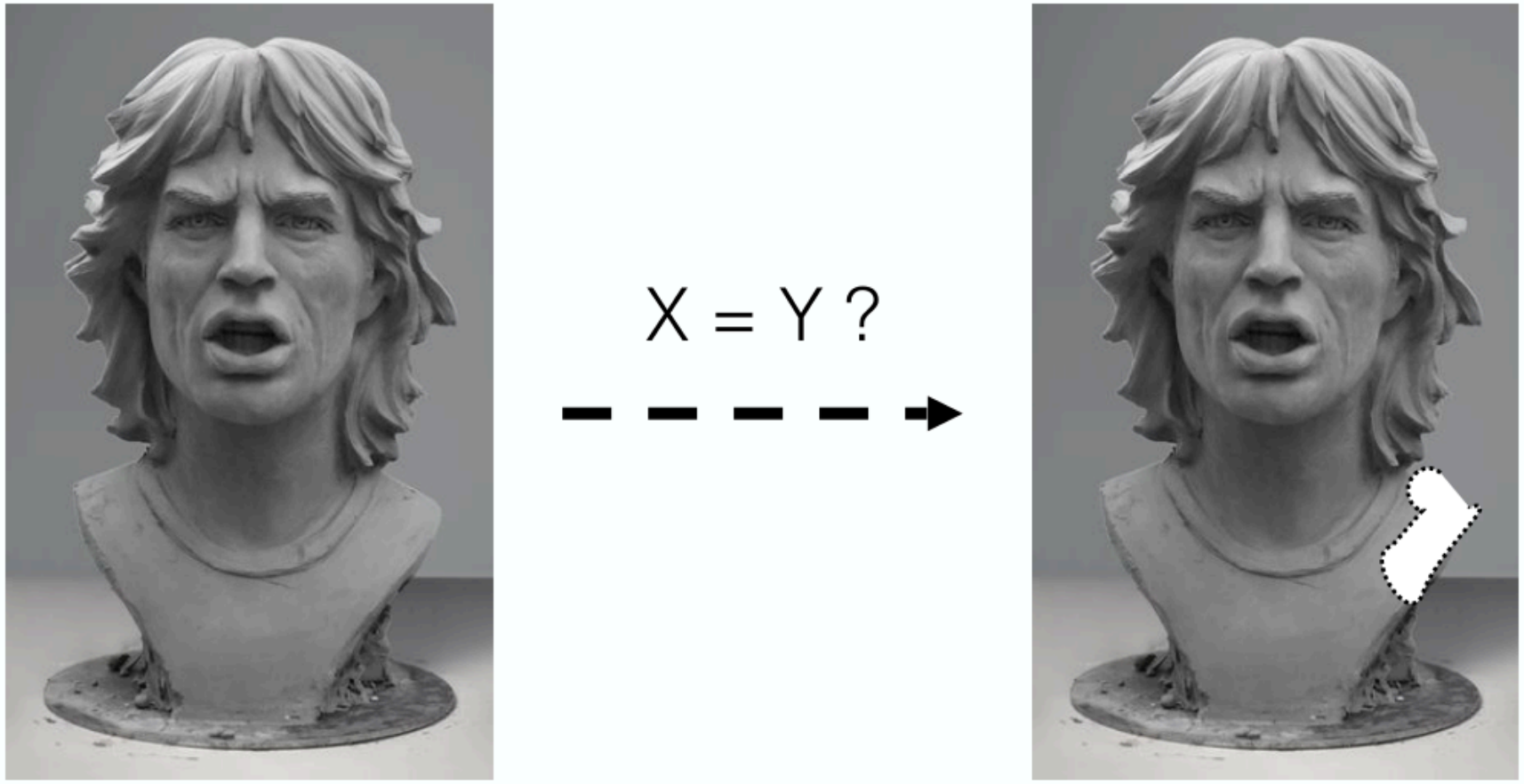


Housenka, motýl vs Lepidoptera

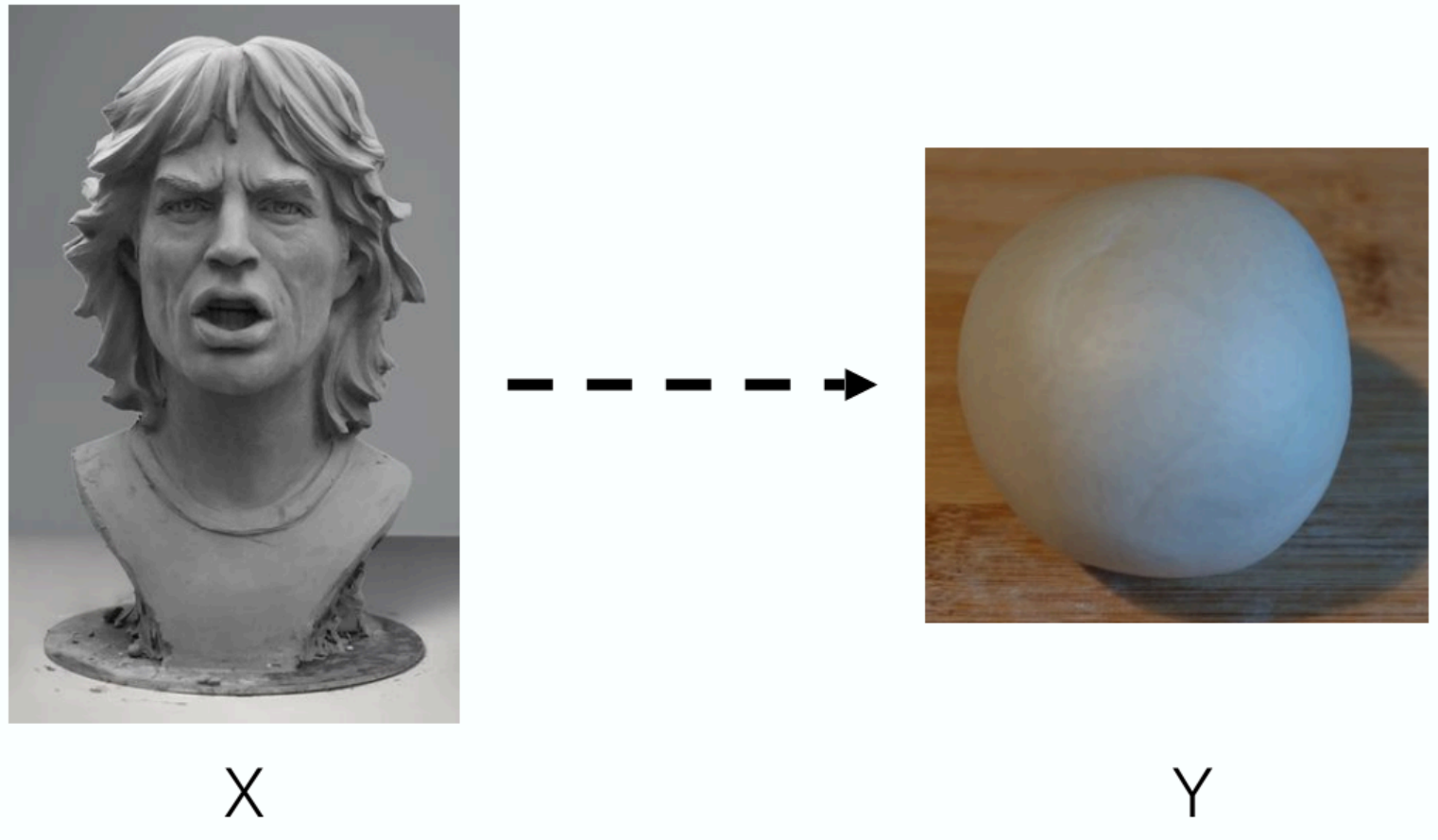
Problematika principu identity



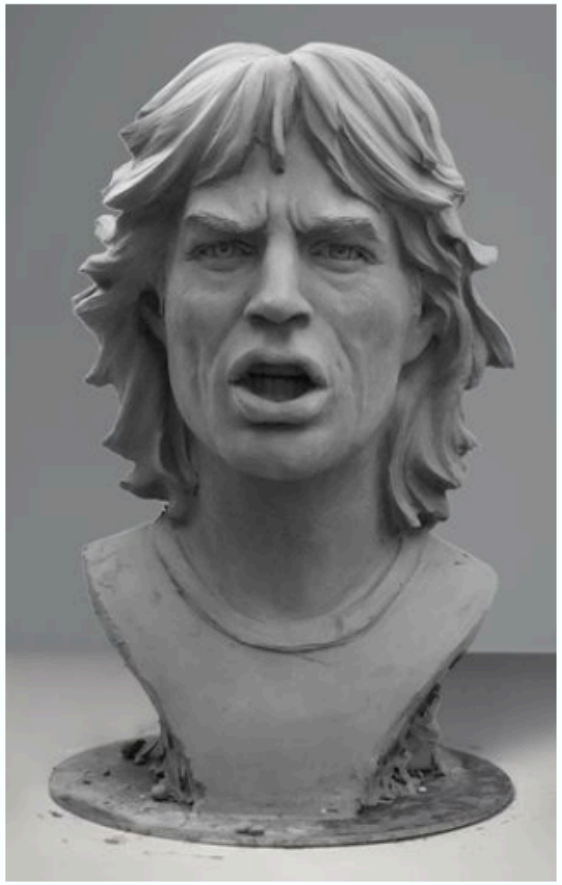
Problematika principu identity



Problematika principu identity



Problematika principu identity



X = Y ?



Problematika principu identity

"Software"

- Softwarový kód: ?
- Program: ?
- Softwarový systém: ?
- Softwarový produkt: ?

Problematika principu identity

"Software"

- Softwarový kód: syntaxe
- Program: ?
- Softwarový systém: ?
- Softwarový produkt: ?

Problematika principu identity

"Software"

- Softwarový kód: syntaxe
- Program: chování
- Softwarový systém: ?
- Softwarový produkt: ?

Problematika principu identity

"Software"

- Softwarový kód: syntaxe
- Program: chování
- Softwarový systém: požadavky
- Softwarový produkt: ?

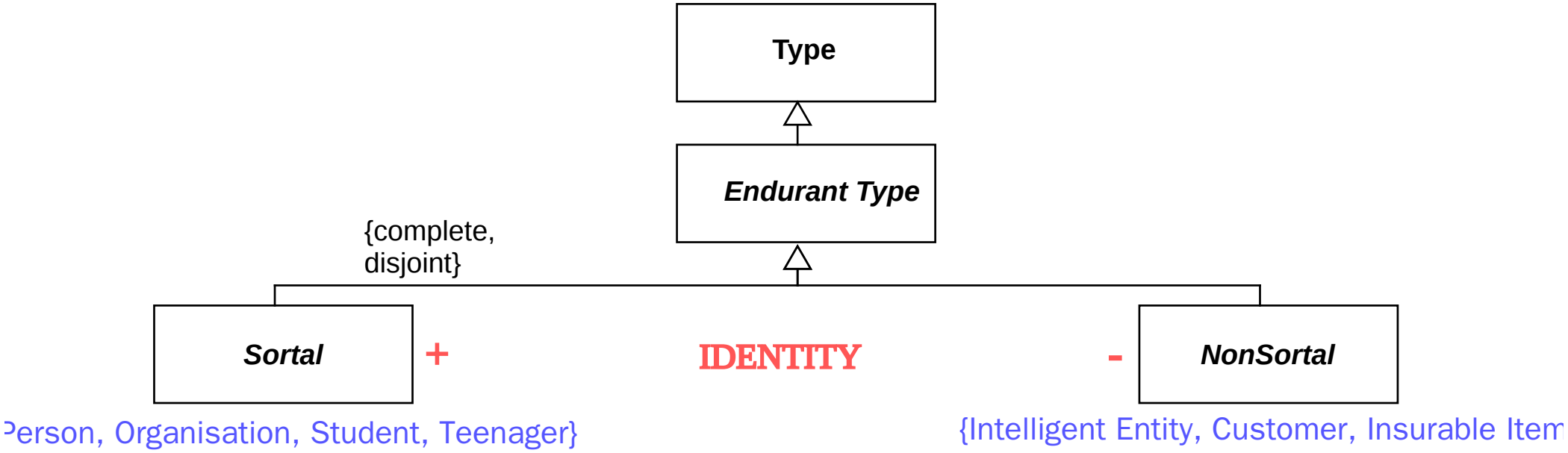
Problematika principu identity

"Software"

- Softwarový kód: syntaxe
- Program: chování
- Softwarový systém: požadavky
- Softwarový produkt: produktová smlouva

Kategorie typů objektů v OntoUML

- Základní rozdělení typů dle principu identity:
  - **Sortal Type** = Má (nějaký) princip identity, tj. referenci a kvantifikaci.
  - **Non-Sortal Type** = nemá princip identity
  - pomůcka: představí si každý (zhruba) "tu samou věc"?



2. Proměnlivost: Modální logika

- Modální logika je rozšířením predikátové logiky.
- V predikátové logice máme kromě spojek výrokové logiky **kvantifikátory**:
  - $\exists$ = **existenciální**: existuje alespoň jeden prvek, že ...
  - $\forall$ = **univerzální**: pro všechny prvky platí, že ...
- Modální logika pracuje s různými *světy*, resp. *situacemi* (konfigurace reality v čase a prostoru), s využitím kvantifikátorů:
  - $\Diamond$ = **možnost**: v některém světě (alespoň jednom) platí, že ...
  - $\Box$ = **nutnost**: ve všech světech platí, že ...

Modální logika - příklady

- $\Diamond Ucitel(robert)$ (pravda)
  - Robert je nyní učitelem na FIT, existuje takový *svět*.
- $\Box Ucitel(robert)$ (nepravda)
  - Robert nebyl učitelem před 30 lety, existuje *svět* kde tvrzení neplatí.
- $\Box x \in Osoba : Muz(x) \vee Zena(x)$ (pravda)
  - Každá osoba je vždy a všude muž, žena, nebo obojí.
- **Modální logika nám umožňuje rozlišovat mezi neměnnými klasifikacemi a relacemi versus klasifikacemi a relacemi, které se mohou měnit v závislosti na kontextu**.

Rigidita - rigidní typy

- **Rigidita** = neměnnost, neschopnost přizpůsobit se změnám
- Typ T je **rigidní** pro každou instanci x, právě tehdy, když je **nutně** ($\Box$) instancí T.
- Pokud je tedy x instancí T v nějakém světě w, tak musí být x instancí T v každém možném světě w'.

$$R^{+}(T) =_{def} \Box(\forall x : T(x) \implies \Box(T(x)))$$

Například $T := Person$:

$$R^{+}(Person) =_{def} \Box(\forall x : Person(x) \implies \Box(Person(x)))$$

Rigidita - anti-rigidní typy

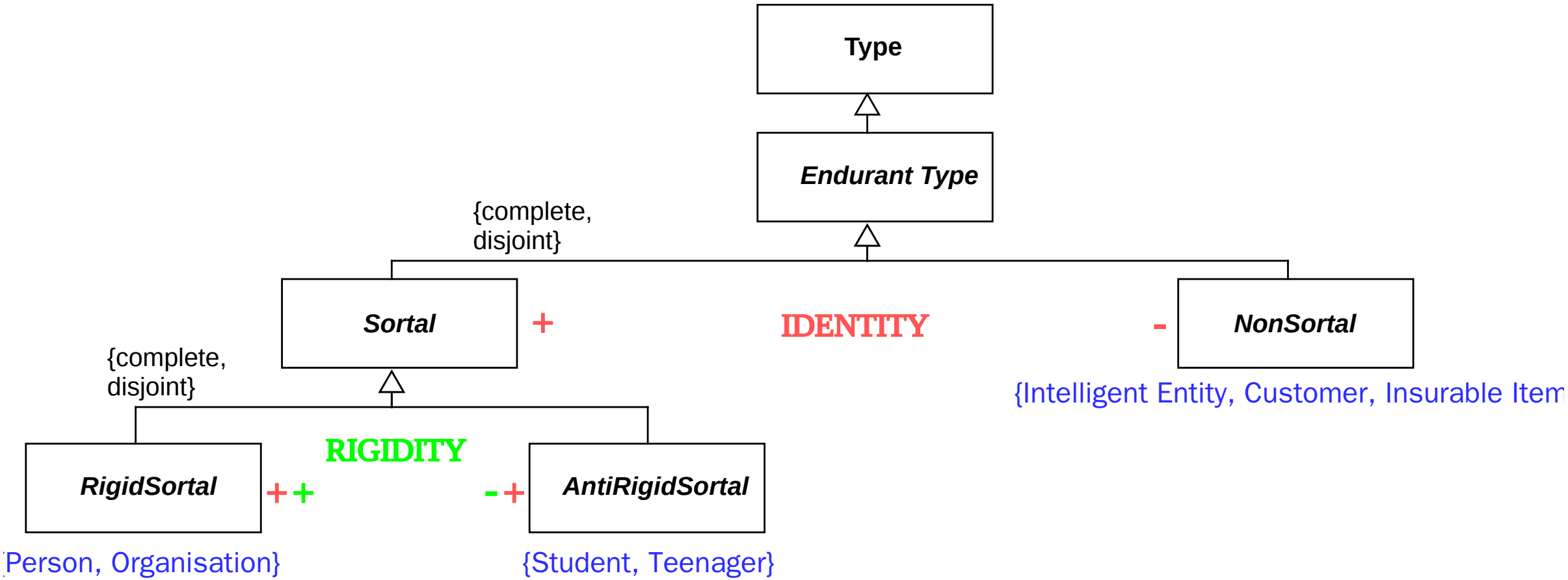
- Typ T je **anti-rigidní** pro každou instanci x, právě tehdy, když je **možné** ($\Diamond$), že x nemusí být instancí T.
- Pokud je tedy x instancí T v nějakém světě w, tak je možné, že existuje nějaký svět w', kde x **není** instancí T.

$$R^{-}(T) =_{def} \Box(\forall x : T(x) \implies \Diamond(\neg T(x)))$$

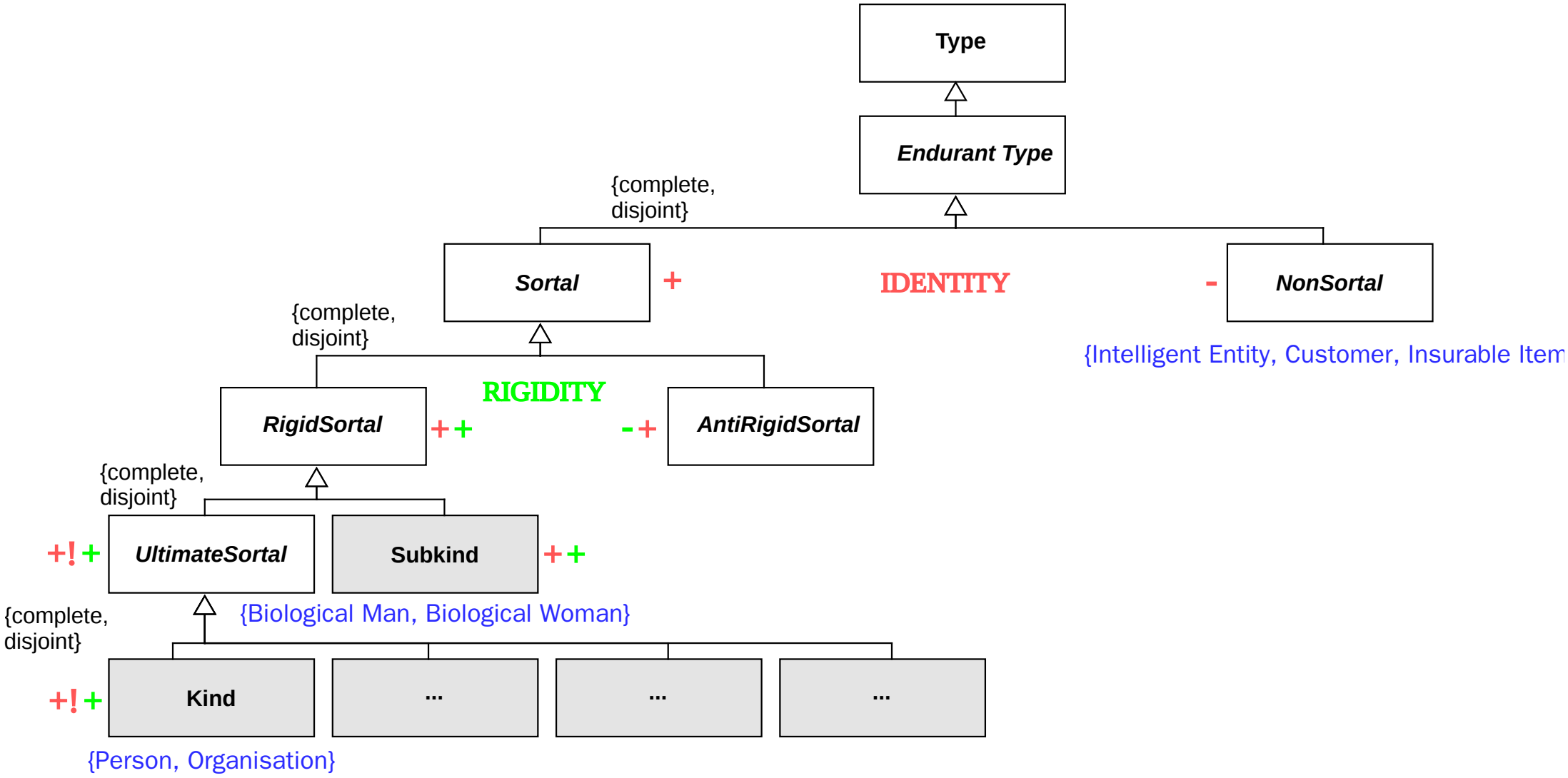
Například $T := Student$:

$$R^{-}(Student) =_{def} \Box(\forall x : Student(x) \implies \Diamond(\neg Student(x)))$$

Kategorie sortálních typů



Kategorie sortálních typů



Rigidní sortály: «Kind» a «SubKind»

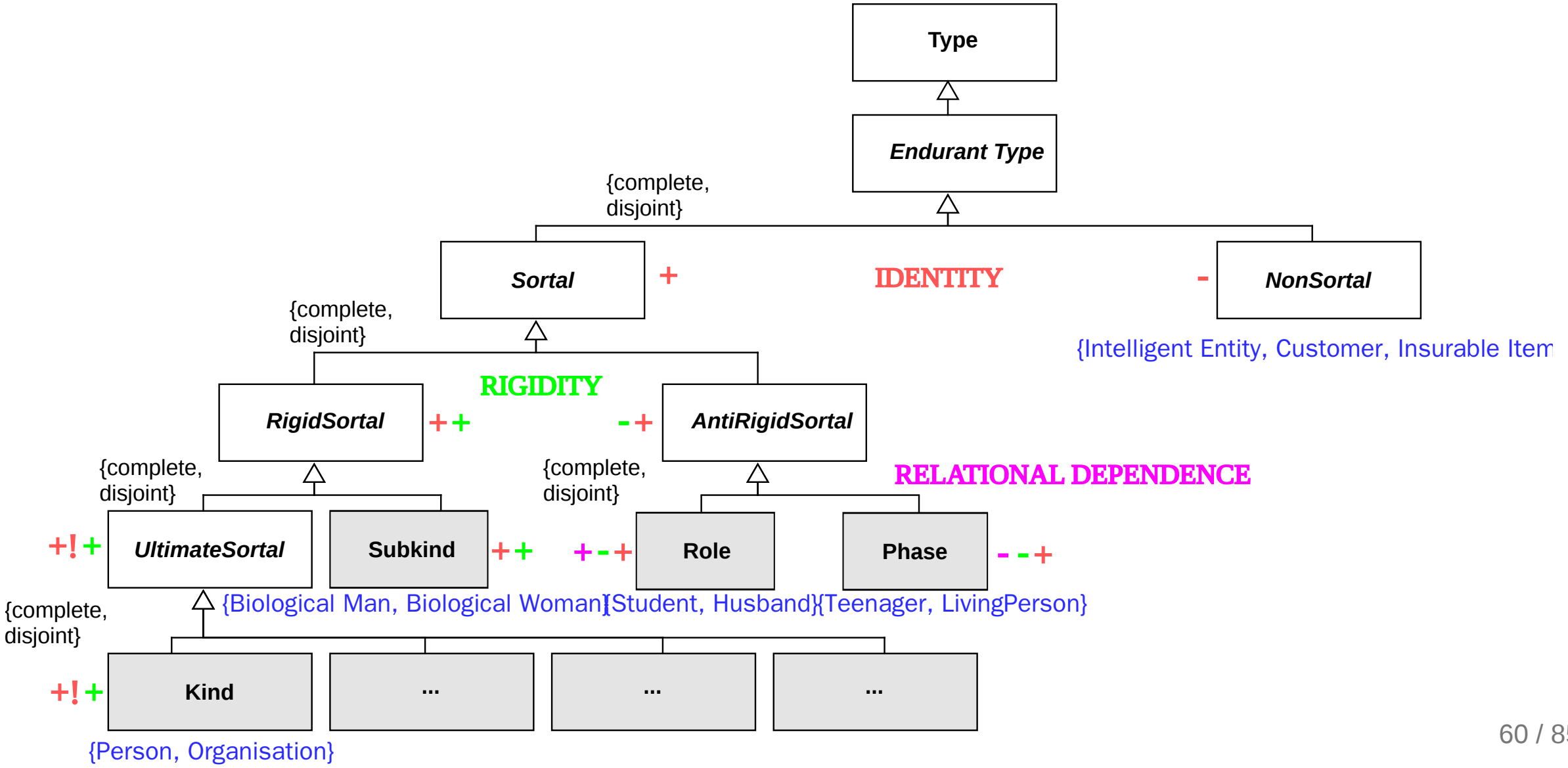
- Mezi *Kind* (typ) a *SubKind* (podtyp) je jemný rozdíl, který UML nerozlišuje, OntoUML však ano.
- V OntoUML je klíčová problematika identity (viz Sortal vs. Non-Sortal).
- *Kind* poskytuje ontologickou identitu, zatímco *SubKind* nikoliv.
- Jedná se opět o identitu z hlediska našeho vnímání: tj. jsme schopni např. rozlišit obecně dva lidi. Muž či žena potom tento princip identity dědí (podobně Brazilec a jiné národnosti), tj. není rozdíl mezi tím, jak rozlišujeme obecně lidi a jak muže či ženy.
- V tomto pojetí "principu identity" není (minimálně na počátku úvah) důležité, *jak* jedince technicky přesně rozlišíme.

Rigidní sortály: Pravidla

- Každý Sortal nese **právě jeden princip identity**.
- *Kind* poskytuje identitu a nemůže být podtypem jiného sortálu.
- *SubKind*, *Role* a *Phase* dědí identitu přímo či nepřímo od právě jednoho **poskytovatele identity**. Cestou mohou mít více rigidních předků.
- Anti-rigidní typ nemůže být nadtypem rigidního typu.
- Non-Sortal nemůže být podtypem sortálu.

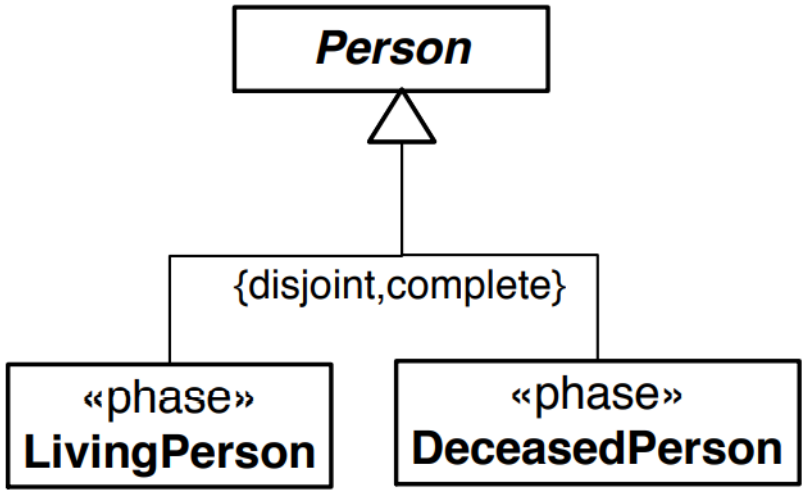
Pozn.: Kind není jediný poskytovatel identity. V přednáškách 3–4 přibudou Relator, Collective, Quantity, Mode a Quality.

Kategorie sortálních typů

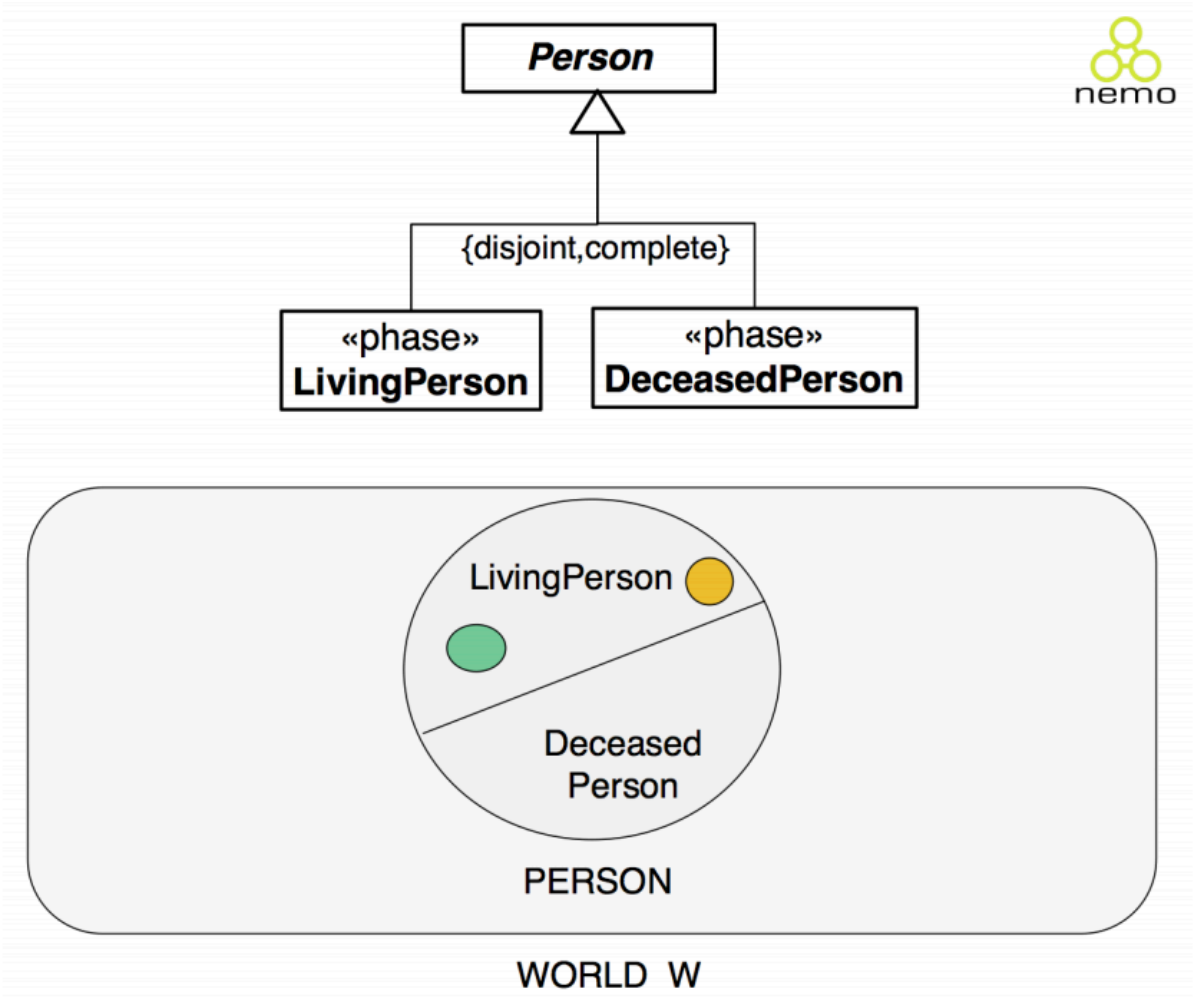


Anti-rigidní sortály: «Phase»

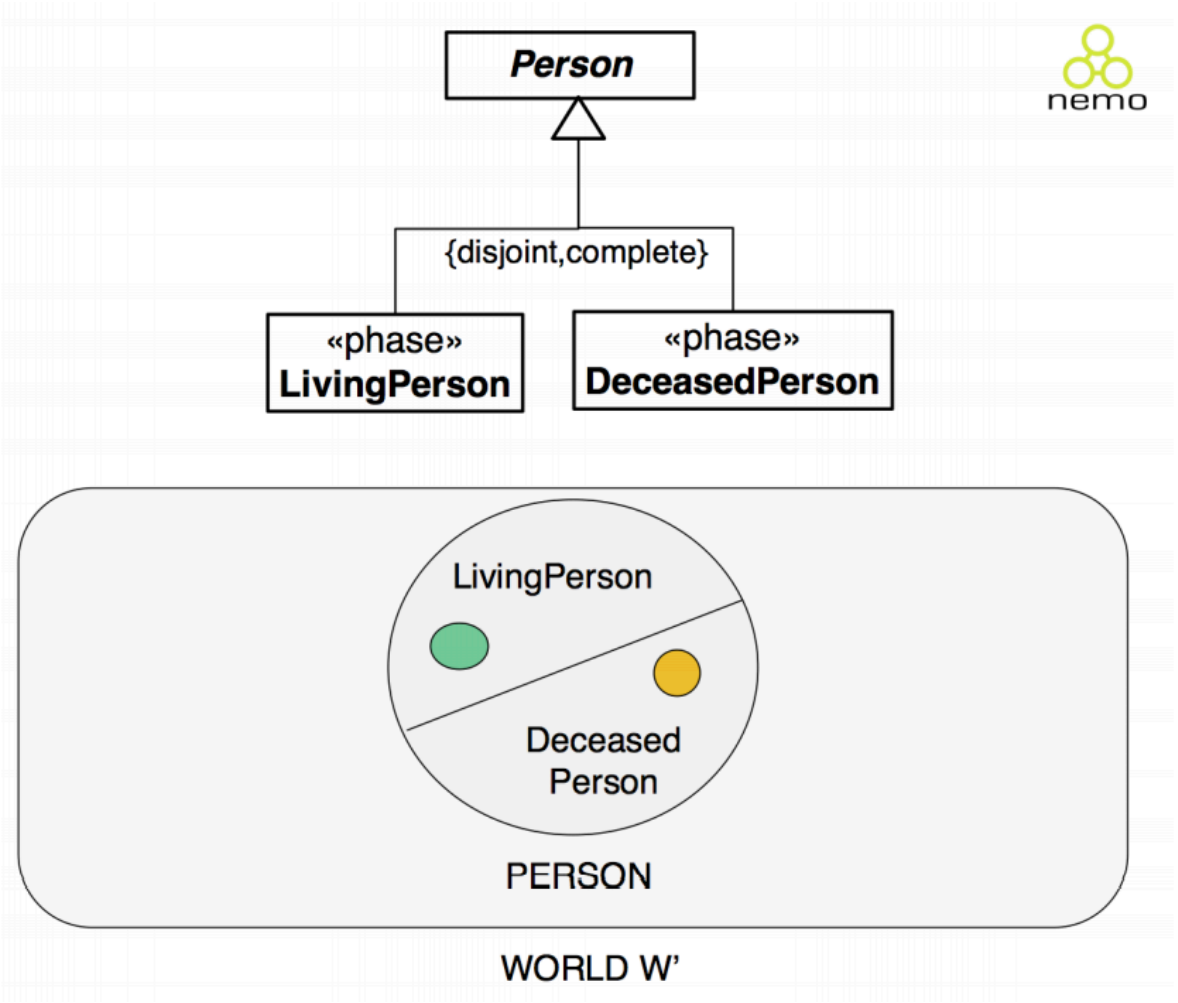
- Anti-rigidní specializace taková, že **podmínka specializace je intrinsická (vlastní, vnitřní)**.
- Tato vlastnost by měla být v modelu explicitně vyjádřena (ve většině příkladů však pro jednoduchost není -- více Přednáška 6).
- Nad fázemi vždy existuje *rozdělení*: **Phase Partition**.
- Phase Partitions jsou vždy `disjoint` a `complete`.



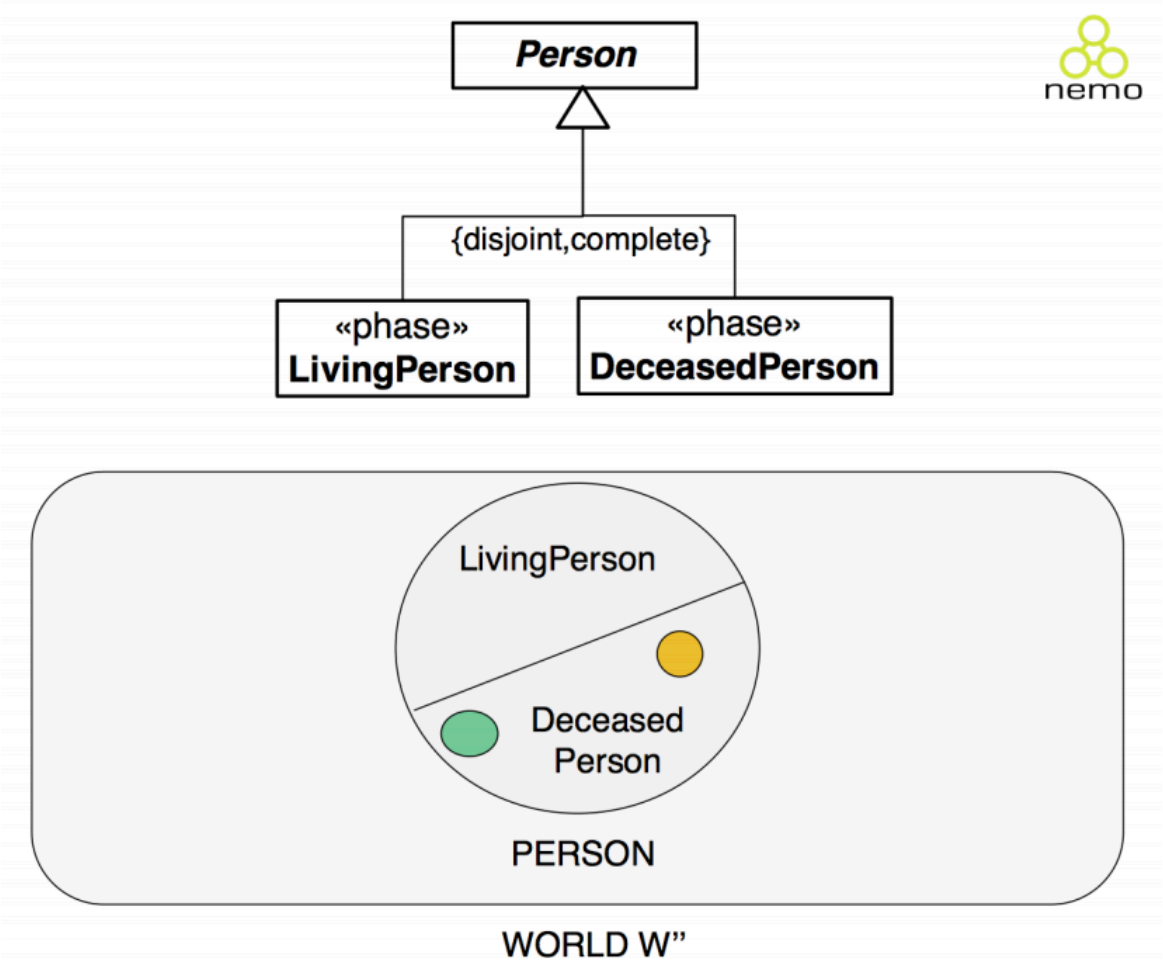
Anti-rigidní sortály: «Phase» - příklad



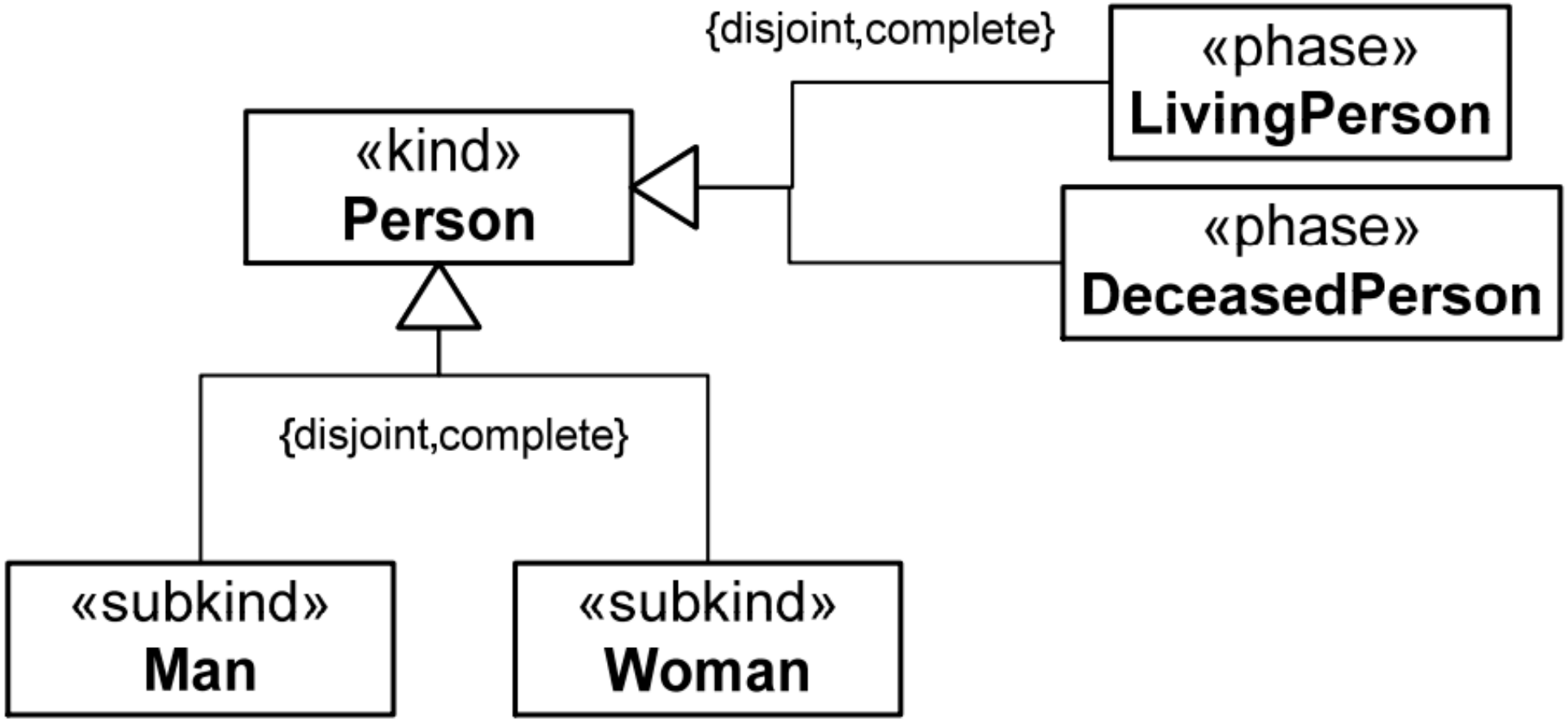
Anti-rigidní sortály: «Phase» - příklad



Anti-rigidní sortály: «Phase» - příklad

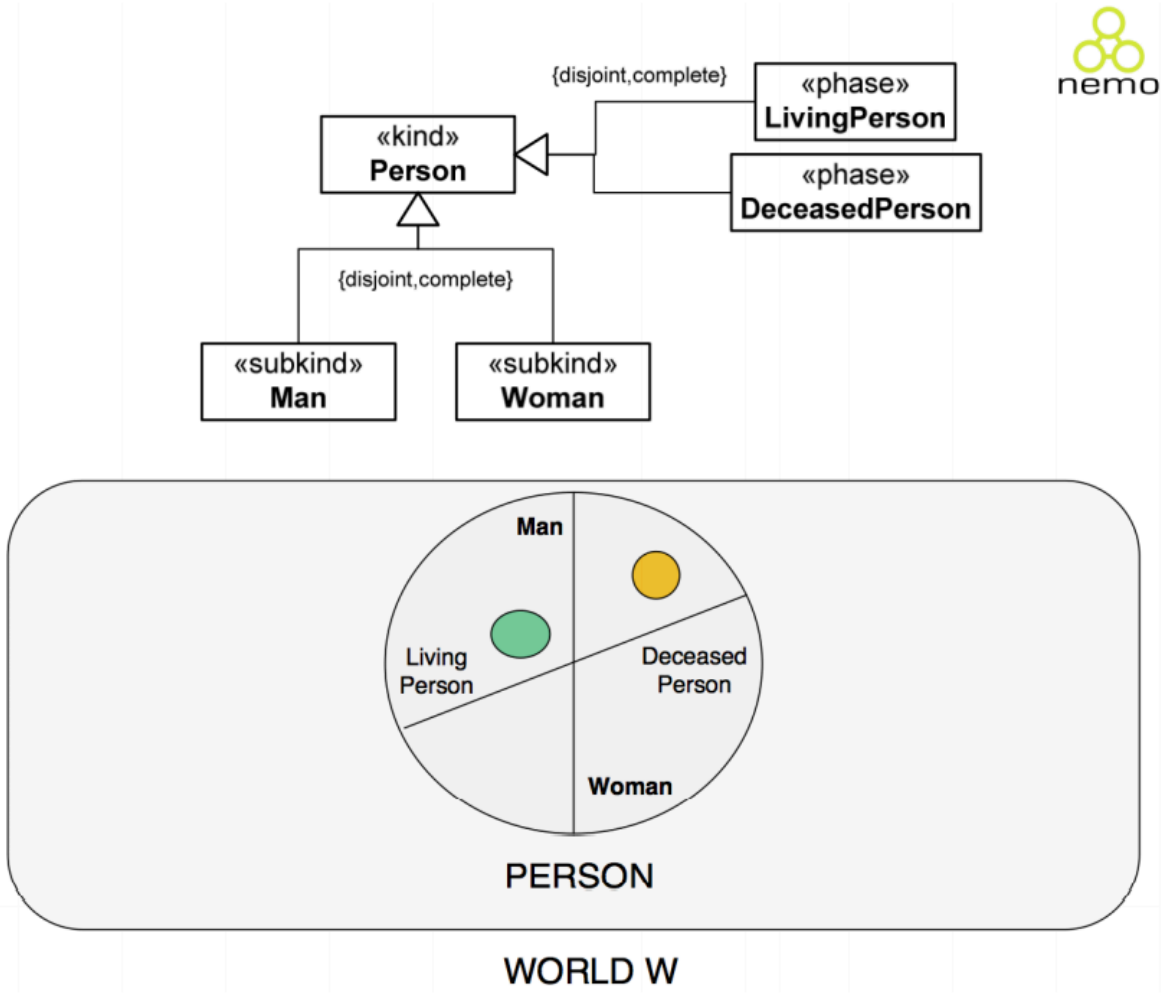


Anti-rigidní sortály: «Phase» - příklad

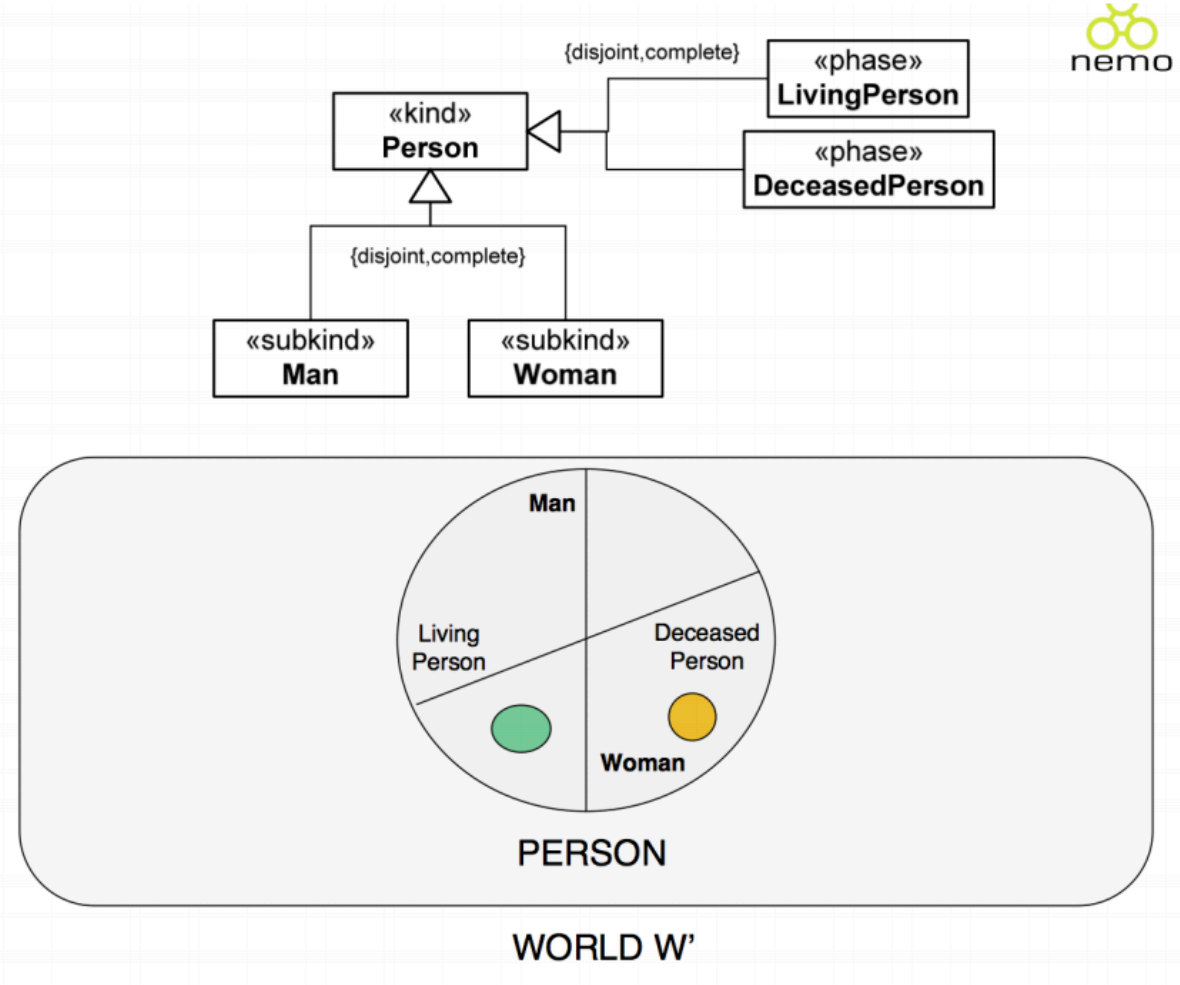


Pozor: rozdíl v chápání instancí UML vs OntoUML

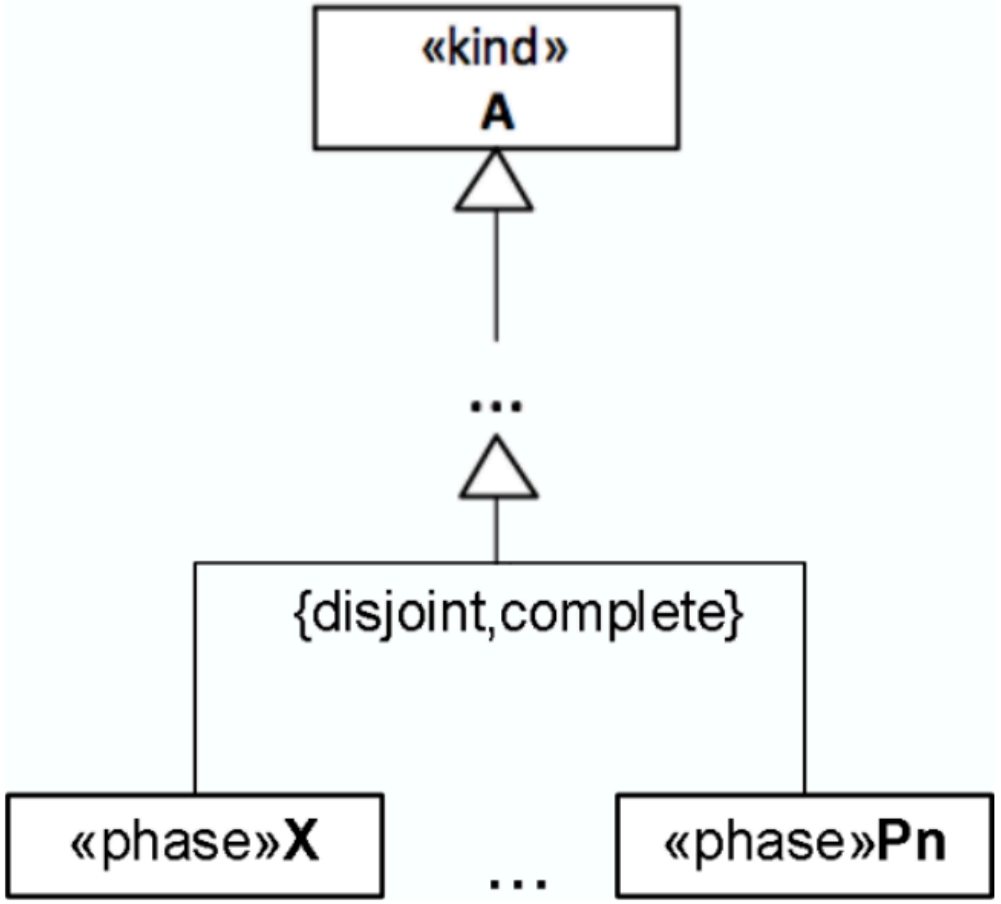
Anti-rigidní sortály: «Phase» - příklad



Anti-rigidní sortály: «Phase» - příklad



Anti-rigidní sortály: «Phase» - návrhový vzor



Anti-rigidní sortály: «Role»

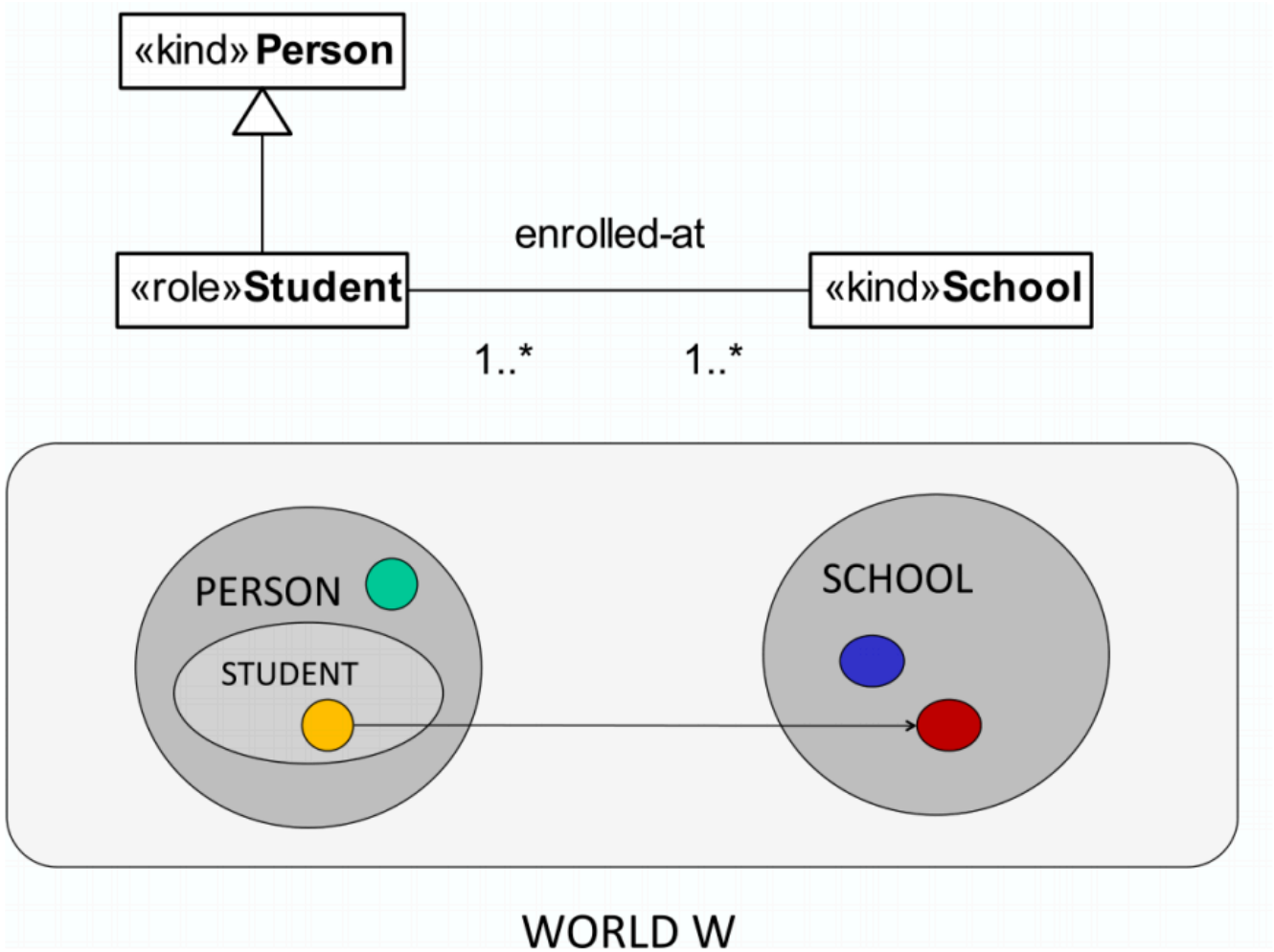
- Anti-rigidní specializace taková, že podmínka specializace je vnější.
- Role podléhá **relační závislosti** (musí mít povinný vztah).
- Typ T je relačně závislý na typu P skrze relaci R, právě tehdy, když pro každou instanci x typy T existuje instance y typu P taková, že x a y jsou v relaci R.

$$D^+(T, P, R) =_{def} \Box(\forall x : T(x) \implies \exists y : P(y) \wedge R(x, y))$$

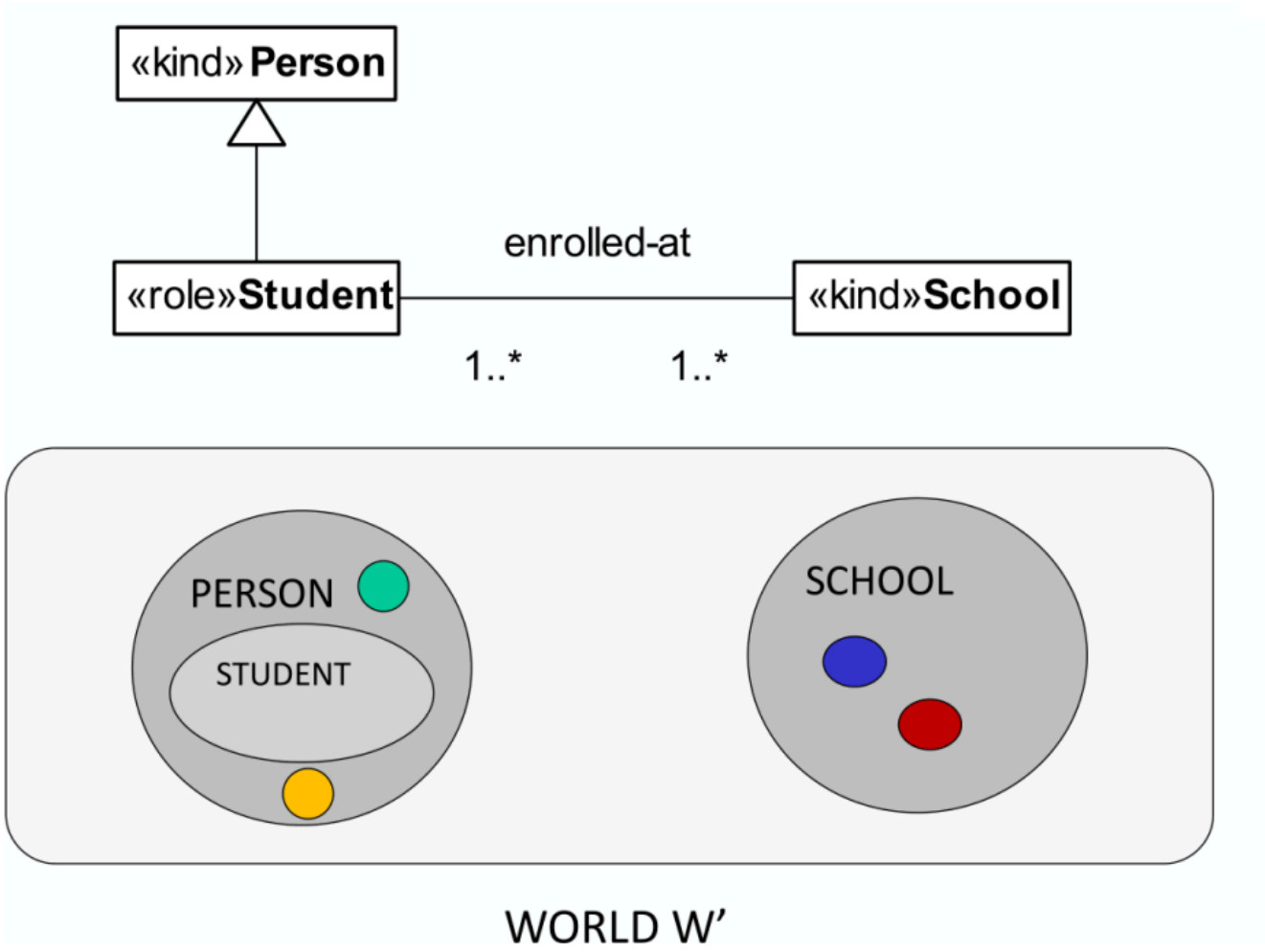
Například:

$$D^+(Student, School, enrolledAt) =_{def} \Box(\forall x : Student(x) \implies \exists y : School(y) \wedge enrolledAt(x, y))$$

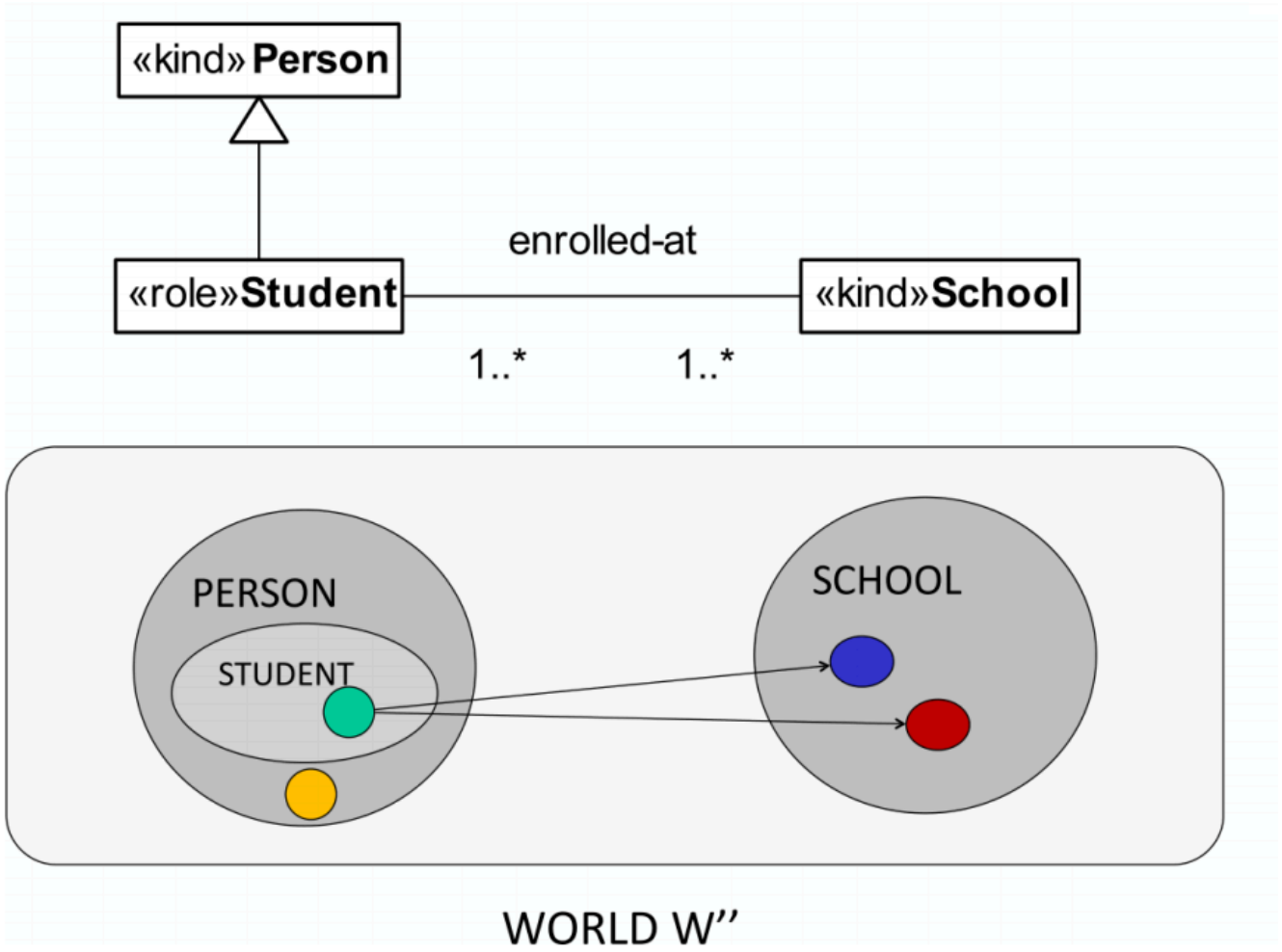
Anti-rigidní sortály: «Role» - příklad



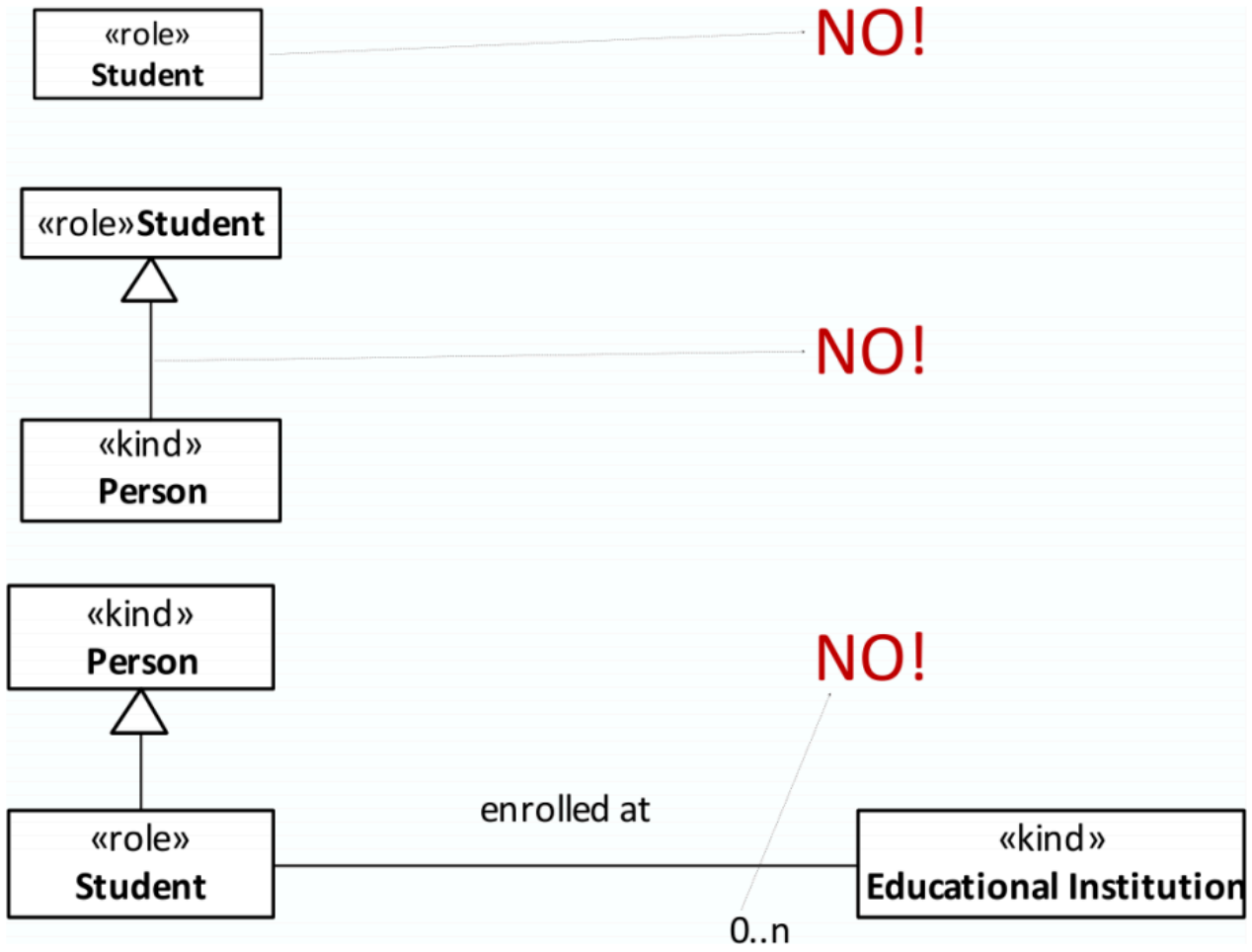
Anti-rigidní sortály: «Role» - příklad



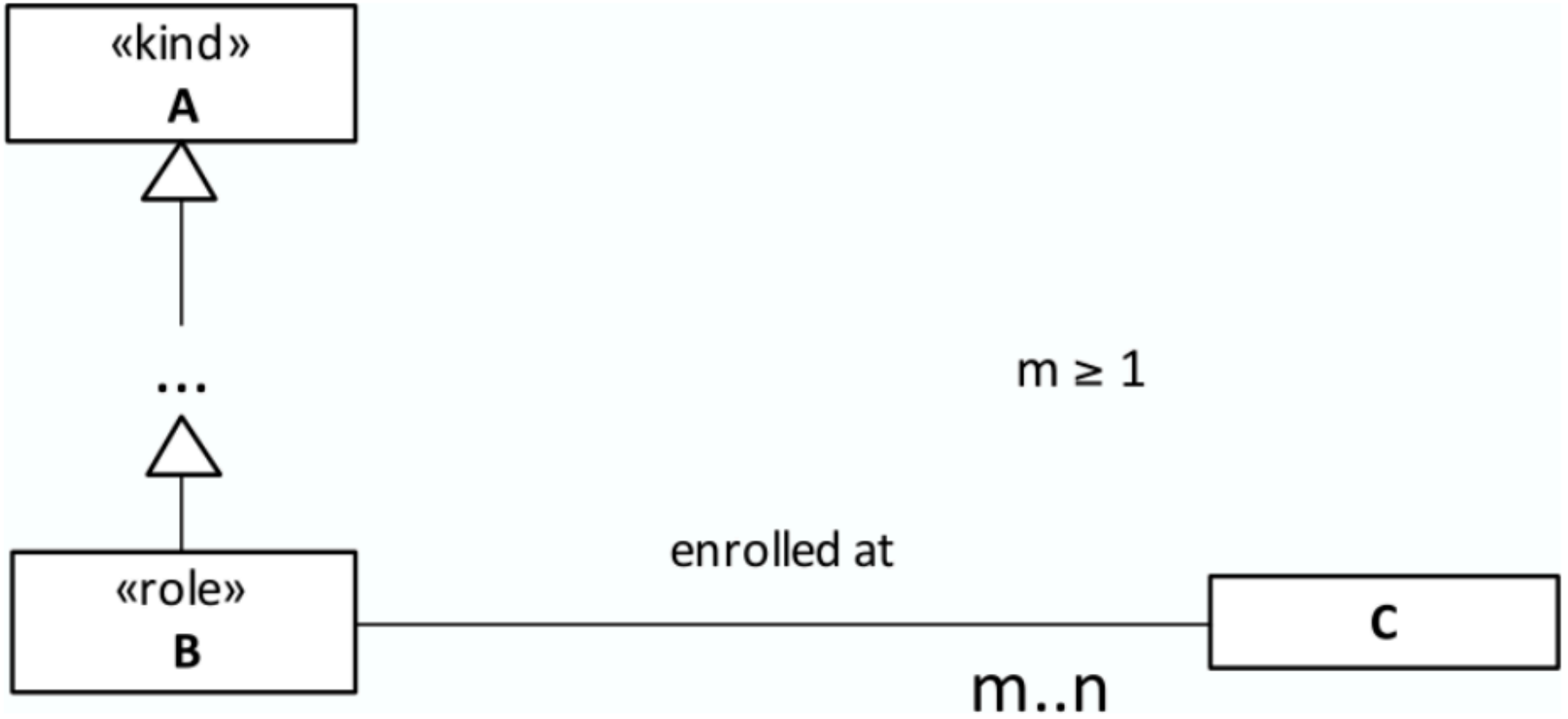
Anti-rigidní sortály: «Role» - příklad



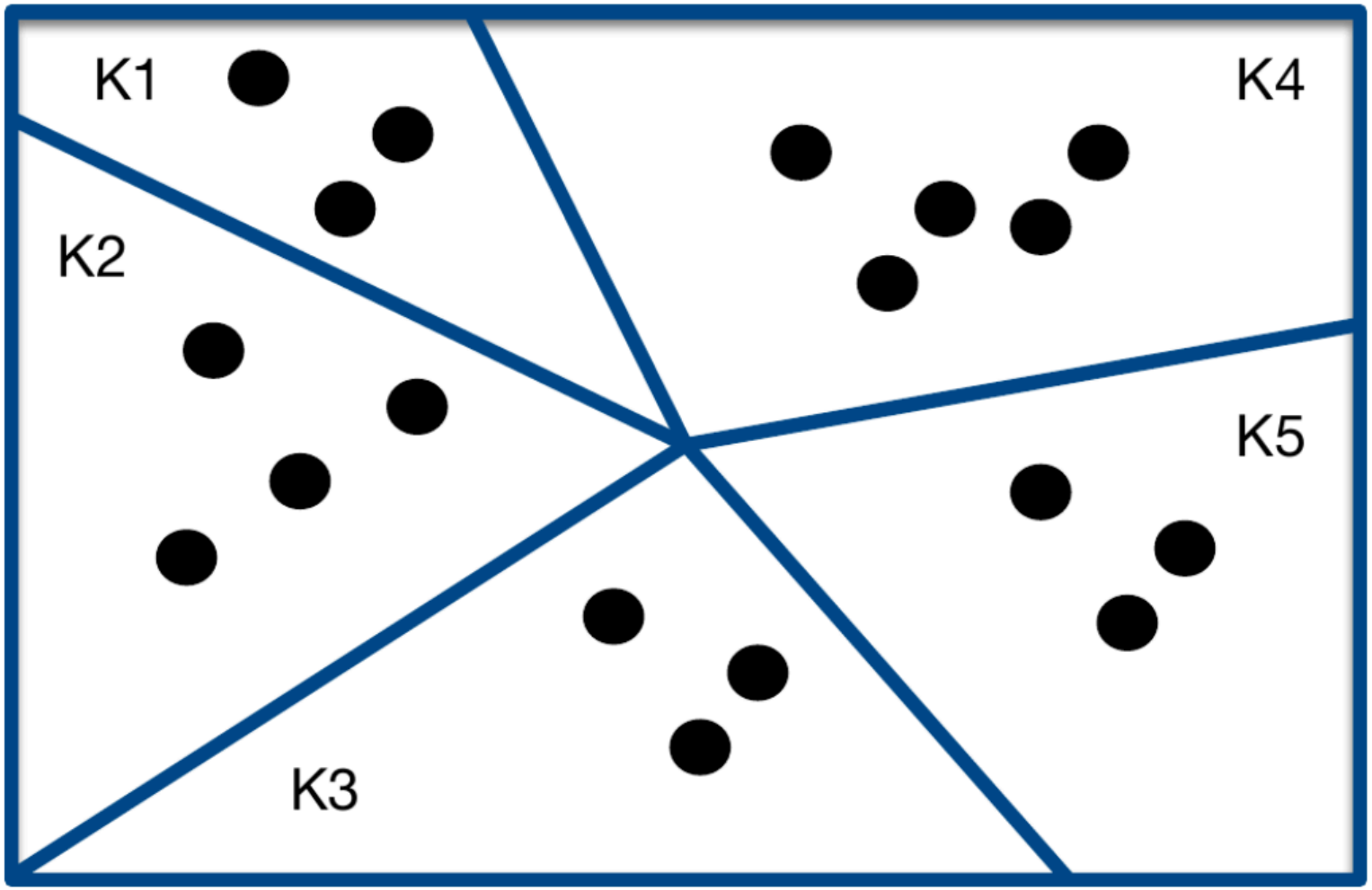
Anti-rigidní sortály: «Role» - časté chyby



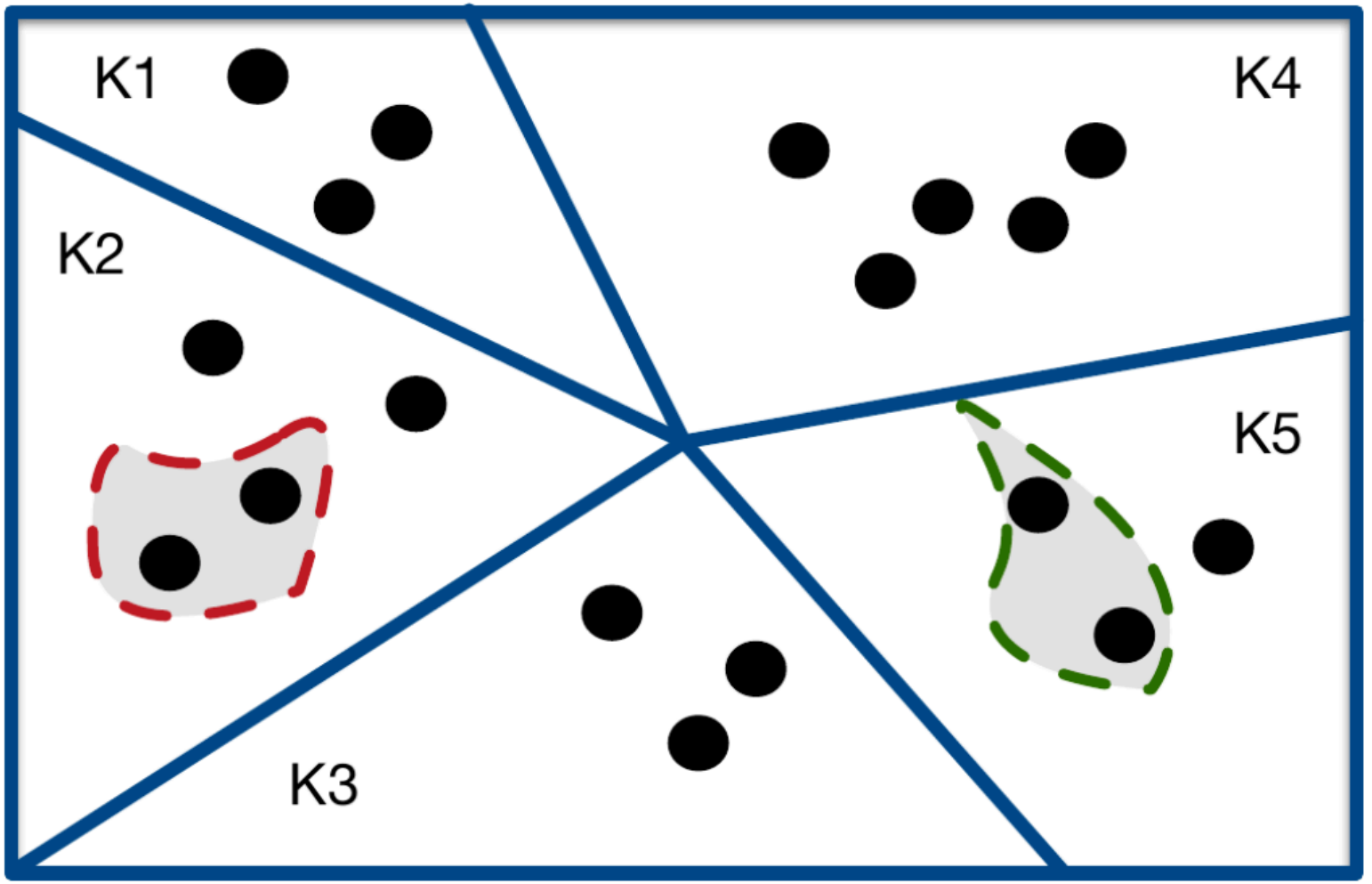
Anti-rigidní sortály: «Role» - návrhový vzor



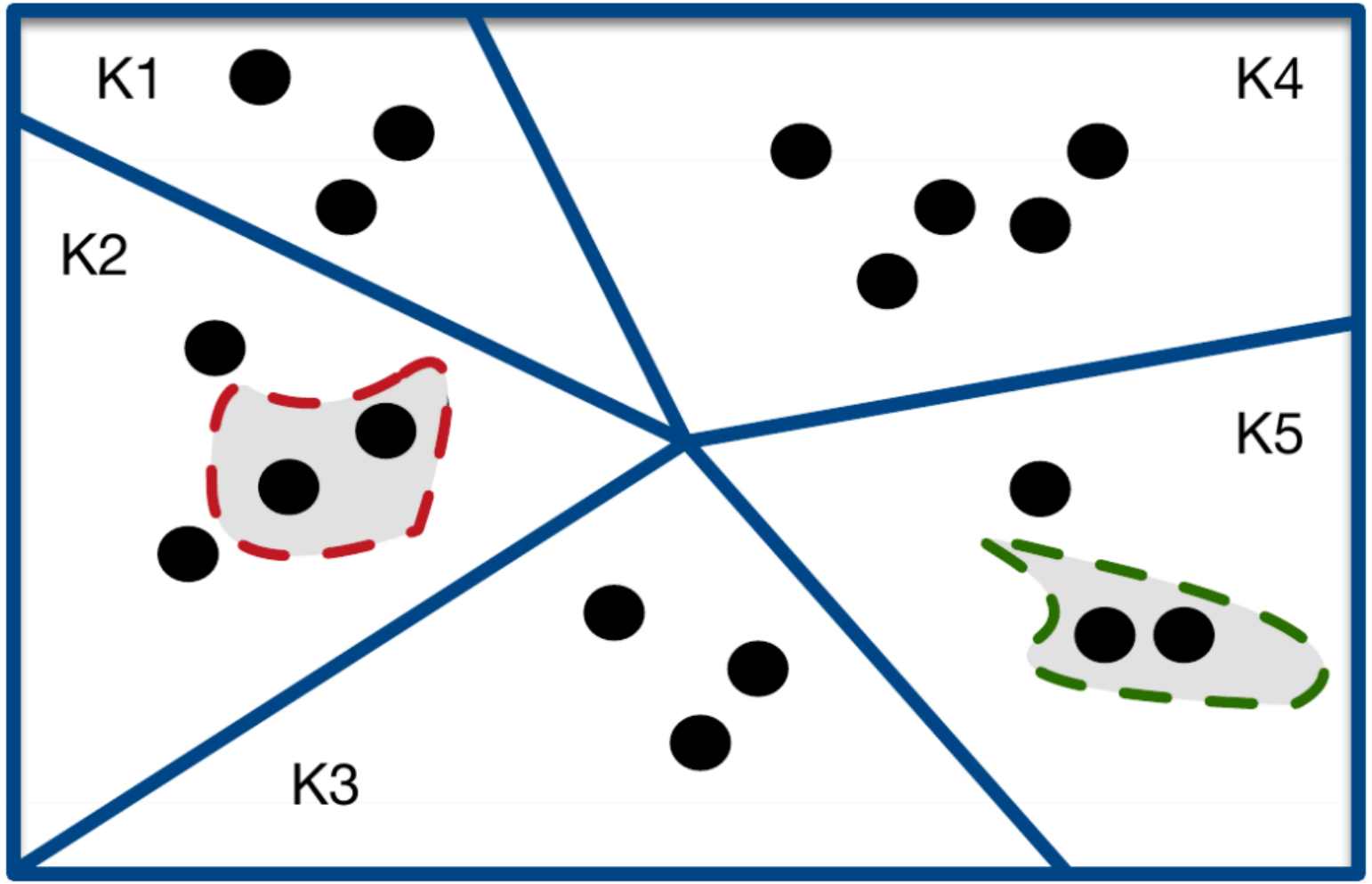
Shrnutí: Kind a SubKind



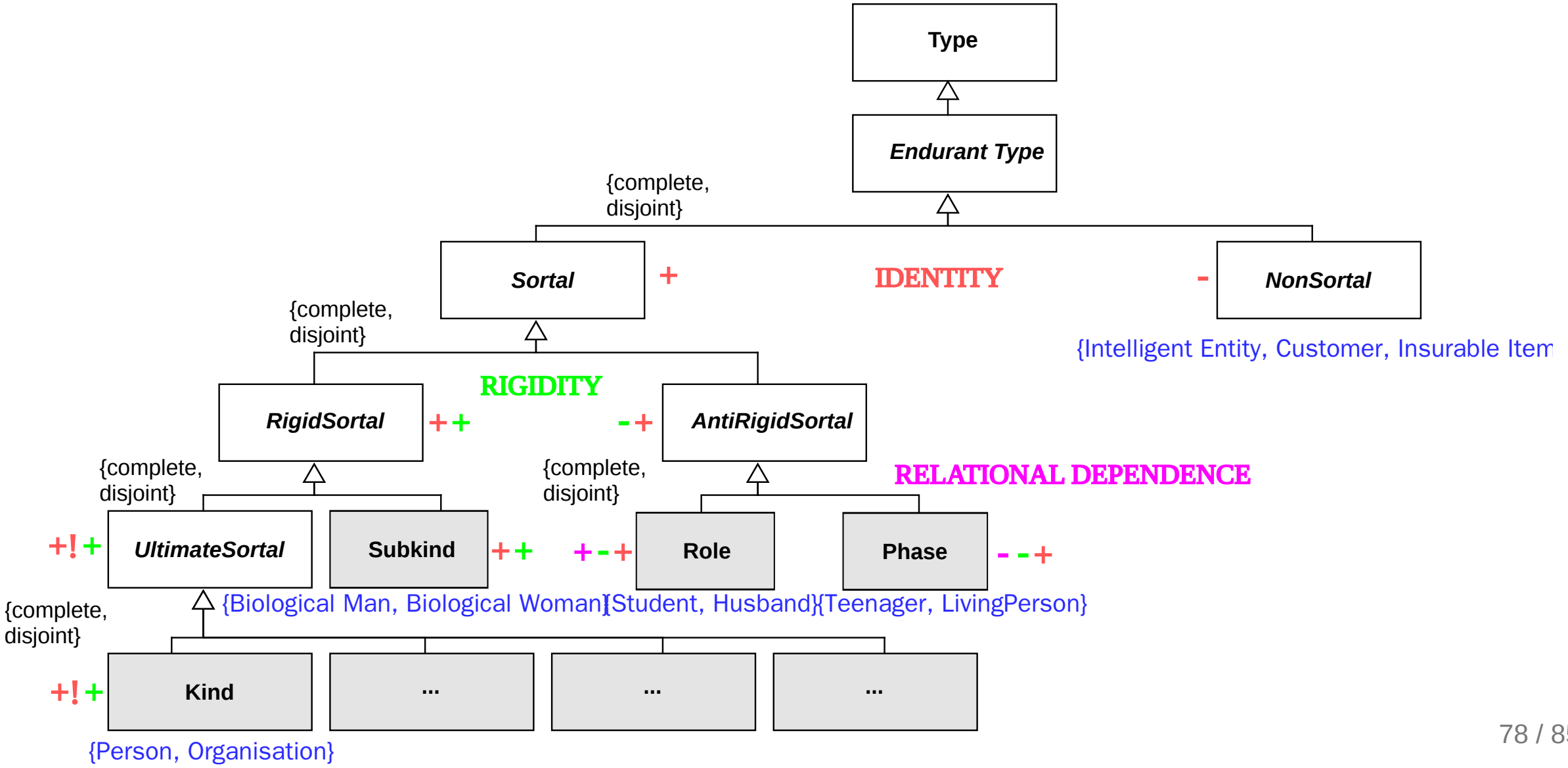
Shrnutí: Role a Phase



Shrnutí: Role a Phase



Shrnutí: Kategorie sortálních typů



Shrnutí: tabulka sortálních typů

Kategorie typu	Poskytuje identitu	Nese identitu	Rigidita	Relační závislost
SORTÁL	–	+		
`«kind»`	+	+	+	–
`«subkind»`	–	+	+	–
`«role»`	–	+	–	+
`«phase»`	–	+	–	–
NON-SORTÁL	–	–		

Shrnutí

- OntoUML rozlišuje řadu typů entit, které se liší svými vlastnostmi.
- Hlavním klíčem dělení je **ontologická identita**, která umožňuje rozlišovat a počítat entity (reference a kvantifikace).
- Analogická problému ekvivalence v programování.
- OntoUML pracuje v modální logice, kterou používá na rozlišování neměnných (Rigid) a měnících se (Anti-Rigid) typů.
- Přesné definice jednotlivých typů implikují pravidla vzájemných vazeb.

Časté otázky

„Můžeme mít se spolužákem dva různé modely a oba správně?“

Ano: stejnou doménu lze modelovat pro různé účely a s různou podrobností. Rozdíly ale musíme obhájit pomocí definic, předpokladů a otázek, na které má model odpovídat. „Každý to vidí jinak“ neospravedlňuje vnitřní rozpor.

Časté otázky

„Proč řešíme identitu tak složitě? Nestačí každé věci přidělit UUID?“

UUID nám umožní jednoznačně odkázat na záznam. Nevyřeší však, zda dva záznamy popisují tutéž věc ani zda po výměně motoru stále evidujeme stejné auto. To jsou doménová rozhodnutí, která musí technická identifikace respektovat.

Časté otázky

„Když někdo zůstane studentem celý život, není Student vlastně rigidní typ?“

Není: anti-rigidita říká, že daný člověk může přestat být studentem a zůstat týmž člověkem. Nevyžaduje, aby k tomu skutečně někdy došlo. Rozhodujeme podle možnosti změny, nikoli podle pozorované délky členství.

Časté otázky

„Proč je Student role a Dítě fáze, když obojí může časem skončit?“

Rozhodující je důvod klasifikace. Studentem je člověk díky vztahu ke vzdělávací instituci; dítětem je v použitém příkladu podle věku. Samotné označení „dočasný stav“ tedy k výběru stereotypu nestačí.

Konec přednášky